Annex IX — Public Handshakes and Contract-Bound Web Interaction

BEAP™ / WR Desk™ / Optirando™ Canonical Specification — standalone Annex. **Version 2.0** (see §IX.24).

Document status: Normative unless explicitly marked as informative.

Applies to: the initiation layer of public BEAP™ entry (code-based entry on all surfaces, invitation-based code distribution, transport-privacy tiers), the WR Execution Client on mobile and desktop platforms, the WR View as a Presentation Surface (Annex V), the Public Handshake, the Site Manifest and its change classes, WR Links, WCR recordings executed against contract-bound web representations, the automation execution classes and the Finalizer consequence boundary (IX.8.3–IX.8.8), the publisher-side normalization pipeline operated on Optirando™ infrastructure, and — from v2.0 — the generalized execution model of the WR Trusted Execution Domain: the Finalizer execution lifecycle, consent mechanism classes, the PoAE™ record and Execution Receipt, verification tiers, and the Transaction Envelope for computer-use (IX.16–IX.23).

IX.0 Status, Scope, and Relationship to the Canon

This Annex specifies the initiation layer — the mechanism by which Public Handshakes come into existence on any surface: code-based entry, invitation-based code distribution, formation with hash-pinned consent, identity lock, transport-privacy tiers, and escalation — and contract-bound web interaction: the mechanism by which a publisher's web presence is transformed into a cryptographically declared, hash-bound, passively rendered representation — the WR View — and by which every content element, navigation, dynamic value, link-triggered function, and automation available to the operator within that representation is confined to what a verified publisher declared in advance under a Public Handshake. It further specifies the WR Execution Client: the governed runtime that forms Public Handshakes, enforces the contract, renders WR Views, and executes WCR recordings against them. From v2.0, it additionally specifies the execution semantics of the WR Trusted Execution Domain (IX.16–IX.23), per the scope extension stated in IX.16.1.

This Annex does not redefine wire formats, capsule serialization, cryptographic primitives, transport, PoAE™ execution semantics beyond the extension of IX.16.1, or the artifact and manifest machinery. It depends on, reuses, and does not weaken:

- **Annex II** — the non-pre-satisfaction rule (§II.2.5), graceful degradation (§II.2.8), and the WR Preparation trigger model (§II.6.5);
- **Annex III** — the mutation class taxonomy and its assignment-table extension rule (§4.5) and the Cloud Abstraction Boundary (§7);

- **Annex V** — reconstruction-based display as the universal presentation path; the WR View is a Presentation Surface in the sense of Annex V and every Presentation Surface rule applies to it;
- **Annex VII** — the artifact and manifest machinery (§6), the verification chain (§7), publisher attestation (P0/P1/P2), the grant model (§10: delivery and preparation rights only; no-expiry-until-revoke; unilateral silent revocation), and the execution boundary (VII-10.1);
- **Annex VIII** — strict passivity of the rendering target, hash-addressed resource resolution with zero render-time network activity, the pinned deterministic feature subset, WebP as the pinned raster serialization, and presentation integrity (VIII.7); the WR View is governed by the same rendering discipline.

The core philosophy of the Canon applies without exception: **the system prepares, the human authorizes, the system records cryptographic proof.** Nothing in this Annex creates, implies, or pre-satisfies any execution authority (Annex II §II.2.5; VII-10.1).

IX.0.1 Claims Discipline (Normative)

This Annex describes an architecture designed to enable high-assurance web interaction and automation as a long-term objective. It SHALL NOT be represented — in specification text, product text, or derived material — as already constituting a certified, formally verified, or “High Assurance” system. “Secure Browse” (IX.9) names a policy mode and a target tier, not a certification. Statements of security properties in this Annex are of two kinds and are marked accordingly: properties that hold by construction of the specified mechanism (e.g. “an undeclared resource has no network path at render time”), and properties that are objectives requiring implementation, testing, analysis, and audit before they may be claimed (e.g. resistance of the client’s parser surface to malformed input). Marketing and documentation derived from this Annex SHALL preserve this distinction.

IX.1 Motivation, Problem Statement, and Threat Context (Informative)

IX.1.1 The Problem — Trusting the Unknown Publisher

The problem this Annex exists to solve can be stated in one sentence: an operator should be able to scan, or manually enter, a completely unknown WR code — published by a party the operator has never encountered — without having to trust that publisher, and nevertheless be safe, because only the cryptographically agreed execution can occur.

No mainstream mechanism provides this today. Scanning an unknown QR code or following an unknown URL hands the publisher the full capability surface of a browser: arbitrary script execution, arbitrary resource loading, fingerprinting and profiling, redirection — and, through that same surface, phishing, malware delivery, and data exfiltration. The trust anchors that exist are aimed at different problems: TLS authenticates a domain, not intent or behavior; platform app review vets installed applications, not the open web; reputation systems presuppose that the publisher is already known. The operator is left to assess, at a glance and on the most sensitive device they own, a party that is by definition unknown — which is no assessment at all. The practical result is that unknown codes are either recklessly scanned or categorically refused, and

both outcomes are failures: the first is the attack surface, the second forecloses the entire class of legitimate spontaneous interactions (the poster, the package, the invitation) this system exists to serve.

IX.1.2 What Must Be Prevented — and What Prevents It

The architecture is designed against the following threat classes. Each maps to the mechanism of this Annex that addresses it; properties marked by construction hold in the sense of the claims discipline of IX.0.1.

Threat	Why it is prevented here
Execution of undeclared or unauthorized code paths	No active content exists in a WR View — script-class capability, timers, and asynchronous interfaces are excluded from the subset and disabled at the engine boundary (IX.6.1, IX.6.7); automation exists only as declared, SHA-256-bound, version-pinned WCR recordings (IX.4.4, IX.8); by construction.
Loading of undeclared external resources	Rendering occurs exclusively from the Verified Local Store under render-time network starvation: an undeclared resource has no route to the network, and its resolution attempt is a deviation (IX.6.2, IX.6.5); by construction.
Data exfiltration	The rendering context holds no network path, no keys, and no consent authority (IX.6.2, IX.10); every transfer target of every automation is declared in the manifest and covered by consent (IX.4.2, IX.8.1); interaction evidence is recorded locally only (IX.11).
Profiling beyond the agreed scope	The Scope Envelope caps data-access classes at contract level (IX.5.1); overlay/interaction patterns are not observable by the publisher (IX.11); transport-privacy tiers protect network metadata, and a publisher can never force a less private tier (IX.3.6).
Prompt or content injection that expands the execution scope	Authority cannot be created by content: the non-pre-satisfaction rule (Annex II §II.2.5) holds without exception; consent attaches to the Scope Envelope and to automation definitions, never to content (IX.5.1); every preview is client-generated from the cryptographically bound definition, never from publisher free text (IX.5.3); slot values are typed, bounded, and rendered inert, never interpreted as markup (IX.6.4). There is no channel through which

Threat	Why it is prevented here
	crafted content could widen what is executable. Beyond that, the model context is itself a contract object (IX.8.6): sources are enumerated and consent-admitted — where every admitted source is typed and instruction-incapable, injection is not mitigated but unaddressable by construction; where a declared source carries free text, that fact is consent-covered, and residual influence within the agreed space is bounded by Decision-Space narrowness, Plausibility Verification (IX.8.7), and the preview at the consequence point.
Malware execution	Nothing delivered executes: strict passivity excludes all executable content classes (IX.6.1); media are upstream-normalized derivatives verified hash-before-decode, so no original byte structure reaches the device (IX.6.3); invitation channels carry only human-readable codes, never capsules or executable attachments (IX.3.1).
Handshake formation or authority claimed through a link	No hyperlink, deep link, redirect, push notification, or channel-delivered payload can form, extend, or upgrade a handshake (IX.3.1); entry exists only through the capture methods of IX.3.1 — operator scan, manual entry, or client-side Assisted Code Capture of inert, already-received data — and a captured code, however captured, is untrusted input until the full verification chain of IX.3.4/IX.3.5 succeeds, with formation occurring exclusively on Hash-Pinned Consent. The invitation channel — email, print, on-screen display, messaging — carries an identifier, never authority (IX.1.5); by construction.
Unauthorized automations	Every automation is individually consent-admitted with a client-generated preview (IX.5.2); it is Manifest-Version-pinned and suspends fail-closed when the site changes under it (IX.8.1); it executes only on the human consent tap with a PoAE™ record (VII-10.1).
Execution outside the previously agreed contract	Origin binding confines everything to the enumerated origin set (IX.3.4); manifests are monotonic and freshness-bound, closing

Threat	Why it is prevented here
	rollback (IX.4.2); any divergence between displayed, permitted, and performed execution is a deviation with hard stop (IX.6.5, IX.7.2); the append-only evidence log makes the executed state provable after the fact (IX.11).
Hacked code, compromised component, or compromised client used to manipulate effects	The agreement binds the effect, not the code (IX.8.8): every externally observable consequence passes a declared, hash-verified Finalizer that accepts only schema-valid, state-bound, capability-carried, consent-evidenced proposals (IX.8.4, IX.8.5). A compromised path can at most fail to produce the agreed effect — never produce a non-agreed one — under the stated trust assumption of correct Finalizer implementation (IX.12.3).

IX.1.3 Platform Context — Exclusion Instead of Containment

The mobile device is the control center of precisely the operators this system exists to serve — journalists, NGO staff, executives, high-risk individuals: location, contacts, messaging, payments, and telephony converge on one device. Unlike the desktop deployment, where untrusted content is contained (depackaging microVMs, sandbox isolation per the established architecture), a mobile platform offers no kernel-level VM isolation under the deployment's control. A scan of a WR Code® from an unknown publisher, in a naïve design, would be an invitation to render arbitrary hostile web content on the most sensitive device the operator owns — with transient compromise (altered and reverted content) potentially leaving no trace.

The mobile strategy is therefore **exclusion instead of containment**: nothing untrusted is admitted in the first place. The publisher's web presence is not rendered as-is; it is transformed upstream, on Optirando™ infrastructure, into a reduced, normalized, hash-bound representation containing no active content, and the client renders exclusively that representation from a locally verified store, with no network activity at render time. The desktop contains what it cannot exclude; the mobile client excludes so that nothing needs containing. The invariant is the same on both platforms — only untrusted content never executes with authority — the enforcement location differs.

A second, complementary description of the same strategy: WR deliberately transforms dynamic web content into a deterministic execution model (IX.6.7). The open web is, in its native form, an open-ended execution environment — scripts run, timers fire, asynchronous interfaces resolve at unpredictable moments, and network activity can alter the presented state at any time after load. The WR View removes these degrees of freedom rather than supervising them, which is what makes the exclusion strategy tractable: what cannot happen need not be contained.

IX.1.4 The Trust Inversion and the Consequence Boundary

The architecture inverts where trust resides. Trust is not placed in the publisher; it derives from cryptographic agreements (Hash-Pinned Consent, IX.3.4), verifiable manifests (IX.4), constrained execution (IX.6, IX.8), deterministic enforcement at the point of consequence (IX.6.5, IX.8.2;

VII-10.1), and auditable evidence (IX.11). An unknown code becomes safe to enter not because its publisher turns out to be trustworthy, but because nothing beyond the agreed contract is executable — the fix is structural, not evaluative, which is why it works for publishers the operator has never encountered.

The governing boundary can be stated in one sentence: **the free decision space ends exactly where externally observable consequences begin.** Within the agreed contract, flexibility is legitimate and deliberate — reasoning, planning, presentation, and interaction, including AI assistance under the corpus's preparation model, may operate freely inside the declared decision space, and the contract accordingly declares boundaries rather than enumerating every runtime value: typed, bounded slots (IX.6.4); declared action, read, write, and transfer sets (IX.8.1); the Scope Envelope as the outer ceiling (IX.5.1). At the moment a consequential, externally observable action is about to occur, execution becomes fully constrained: only the previously agreed, hash-verified, consent-covered execution is permitted, and it passes through the deterministic enforcement of this Annex. In this frame, intelligence and automation are separate architectural domains — analysis, reasoning, and user interaction on one side; execution of contractually permitted actions on the other. Neither requires the other, either may operate alone, and their combination changes nothing at the boundary: however flexible the path to a proposal, the consequence is gated identically.

Automation is one application of this model, not its definition. The contract governs everything the operator perceives and everything the operator can do: displayed content, resources, navigations, inputs, state transitions, WR Links, media, and automations alike.

IX.1.5 The Link-as-Entry Failure Class (Informative)

Problem. The naïve design for a public entry point is a clickable link: the invitation is the entry. An ordinary hyperlink, however, fuses three distinct functions into a single user action — **presentation** (what the operator sees), **navigation** (where the operator is taken), and **claimed authority** (the implication that the destination is the legitimate publisher). Because all three collapse into one click, the client is never given the opportunity to establish a trust boundary before interaction with the claimed publisher begins. Whatever the destination is, the operator is already there.

Attack surface. Treating a link as the entry point imports, wholesale, the established attack patterns of the open web into the handshake architecture:

- **Publisher and destination spoofing** — a link claims to lead to publisher A and leads to attacker B; nothing in the link itself binds claim to destination.
- **Phishing invitations** — messages visually indistinguishable from legitimate handshake invitations, carrying a fraudulent link.
- **Misleading hostnames** — deceptive domains, subdomains, and URL paths; homoglyph and lookalike hostnames that survive visual inspection.
- **Redirect and redirect-chain manipulation** — the displayed target and the final destination differ by construction of the web; each hop is an opportunity for substitution.
- **Compromised distribution surfaces** — websites, emails, advertisements, QR codes, and messaging channels that present fraudulent links, whether by compromise of the surface or by malicious placement on an honest one.

- **Link substitution and recontextualization** — links forwarded, copied, republished, or embedded outside their original context, severing whatever contextual trust the original placement carried.
- **Social engineering** — urgency, authority, and scarcity pressure applied so that the operator clicks before any independent assessment of the publisher occurs.
- **Malware delivery** — a link is a request for arbitrary content; the destination may deliver exploit content, drive-by payloads, or a hostile page exercising the full capability surface of a browser (IX.1.1).
- **Channel/authority confusion** — the operator conflates the channel that delivered the link (a known contact, a familiar site) with the source of authority for what the link does; the channel's trustworthiness is silently transferred to the destination.
- **Habituation** — the decisive long-term failure: users are trained, across the entire web, to treat a familiar-looking link as sufficient proof of authenticity. Every legitimate link-based flow deepens the conditioning that the fraudulent one exploits.

The email case, specifically. Email deserves explicit treatment because it is the highest-volume invitation channel and the most instructive failure. An emailed link opens every class above at once — credential phishing, attachment- and link-borne malware, display-name and reply-chain spoofing, thread hijacking, homoglyph sender domains, tracking-based profiling, and urgency-driven social engineering. Email does possess authentication machinery: SPF, DKIM, and DMARC cryptographically bind a message to a sending domain. But this machinery illustrates precisely the design flaw this Annex avoids: **verification exists as a hidden path.** The signatures are evaluated by infrastructure and buried in headers; their outcome is surfaced — if at all — as an indicator a specialist can locate and interpret. A technically skilled operator with time can verify a message's origin; the actual population at the point of decision — non-technical operators, under time pressure, on a mobile device — cannot and does not. A security property that must be sought out by the party it protects protects, in practice, almost no one. Worse, the ecosystem's own defenses compound the habituation problem: link-rewriting protections in enterprise mail systems replace every URL with an opaque scanning-service URL, institutionalizing the habit of clicking links whose destination cannot be read at all. And even where DKIM verification succeeds, it authenticates the sending domain — not the publisher's intent, not the destination of any embedded link, and not what will execute there (the trust-anchor mismatch of IX.1.1).

Architectural cause. The deeper error is not that malicious links exist — they always will. The error is allowing the untrusted distribution channel to claim authority: making the invitation channel itself the authoritative initiation mechanism. Once the channel is the entry, every property of the channel — spoofability, redirection, substitution, opacity, social pressure — becomes a property of the entry. No amount of channel-side hardening escapes this, because the channel is by definition outside the client's control.

Control. This Annex eliminates the failure class by separating four phases that the link collapses into one:

1. **Invitation distribution** — MAY occur over any untrusted external channel (print, screen, email, message, advertisement). The channel carries a human-readable code and explanatory text, nothing more (IX.3.1). Nothing distributed executes, establishes authority, grants permission, creates a handshake, or bypasses client-side verification.

2. **Entry** — begins only when the operator deliberately submits the code to the trusted client: by WR Code® scan or by manual entry (IX.3.1). The code acts purely as an identifier and lookup reference.
3. **Verification** — the client resolves the handshake material through the defined verification path and independently verifies publisher identity, cryptographic signatures, domain binding, the allowed origin set, integrity hashes, and handshake metadata (IX.3.4, IX.3.5) — dual-channel, fail-closed.
4. **Consent** — only after verification succeeds is the verified publisher identity, the handshake detail, and the consent interface presented (IX.5.3); consent authorizes exactly the verified and displayed scope, Hash-Pinned.

The governing principle: **distribution is not entry.** And its corollary, normative in IX.3.1: **the code is an identifier and invitation — not authority; authority arises only from successful client-side cryptographic verification.** A hyperlink may at most inform — explain what WR is, or help the operator locate the official client — but it never initiates the trusted flow. A scanned or printed code is not trusted either: a WR Code® is untrusted input like any other until the complete verification chain has succeeded; the scan path is client-controlled and fail-closed precisely for this reason.

This is also the answer to the hidden-verification-path flaw of email authentication: the WR design does not ask the operator to verify anything. Verification is performed by the client, unconditionally, on every entry, and its success is the precondition for the consent interface even appearing. The operator's only security-relevant acts are deliberate ones — submitting the code, and consenting to a verified, client-generated preview. Cryptographic proof of the connection point replaces visual inspection of it; and unlike DKIM, the proof is not an artifact the diligent can excavate but a gate the flow cannot pass without.

Why manual entry is a high-assurance property, not a fallback. Manual code entry is deliberately co-equal with scanning (IX.3.1), because it embodies the separation in its purest form. It creates a hard discontinuity between the external communication channel and the trusted client: no navigation event connects them, so no attacker-controlled flow can be entered by the channel. The operator's deliberate act of transcription is the trust-boundary crossing — an unmistakable, unspoofable, operator-initiated handoff — after which the client, and only the client, controls resolution, verification, rendering, and consent. Where a link race is won by whoever crafts the most convincing click, code entry has no race: there is nothing to click.

Why Assisted Code Capture does not reopen the failure class. From v1.8 onward, the client may itself recognize a WR code without operator transcription — extracted by the depackaging pipeline from a received message, or detected via a Discovery Record on a visited domain — and present a verified Connect offer (Assisted Code Capture, IX.3.1). This automation preserves the four-phase separation because the phase boundaries are drawn around authority, not around effort. In assisted capture the client — never the channel — performs the recognition, on inert, already-received data: for email, on the validated text of the depackaged BEAP™ message, the transformation that renders a received email machine- and human-readable, text-pure, secure, and deterministic by design, with the original artifact never crossing the boundary (blob-inertness). Only the typed, instruction-incapable code token parameterizes the flow; the channel still cannot initiate, navigate, execute, or claim anything. What automation removes is transcription effort, not the trust boundary: verification remains unconditional and fail-closed, a Connect offer appears only for a successfully verified

publisher, in client chrome, with its capture provenance stated, and formation still occurs only on the Hash-Pinned Consent tap — a failed verification suppresses the offer entirely. The decisive contrast with the link therefore survives intact: a link asks for the operator's click **before** verification, on the attacker's presentation; a Connect offer asks for it **after** verification, on the client's presentation of the verified identity and attestation level. The residue is likewise unchanged — an attacker gains at most a lower-friction way to present its own, correctly verified identity — and because assisted capture deliberately reduces friction relative to manual entry, it is policy-governed (IX.9): deployments whose threat model warrants manual entry's full discontinuity can disable or downgrade it, and scan and manual entry are never replaced. Distribution is not entry; detection is not entry either; consent completes entry.

Assisted capture also closes the hidden-verification-path gap from the other side. The same BEAP™ depackaging transformation that yields the validated message evaluates the channel's own authentication standards automatically — SPF, DKIM, DMARC, the sender domain's Discovery Record — carries the verdicts as typed fields of the resulting message, and the Connect offer states them as plain facts, including whether the authenticated sending domain aligns with the verified publisher (IX.3.1, rule 7). What email leaves buried in headers for experts to excavate, the client surfaces automatically at the decision point, in language a non-technical operator under time pressure can act on. The division of labor is deliberate: the WR verification chain answers whether the publisher is who it claims to be and what can execute — structurally, even for a hostile publisher; the channel verdict answers the remaining human question — whether a handshake should be formed from this channel at all — and, under the ratchet discipline, it can only ever add friction, never authority. A passing channel proves nothing about the publisher; a failing one is a fact the operator sees before deciding. The record also follows the message beyond capture: the external-link risk dialog surfaces the identical verdicts with identical alert presentation (IX.3.1, rule 8), so an unauthenticated channel — one where the sender address itself cannot be verified for lack of DKIM/DMARC — is flagged in red wherever the operator is about to act on the message's content, whether that act is forming a handshake or opening a link. One evaluation, one record, every decision point. The same facts also feed the deployment's local Fraud Watchdog™ (IX.3.1, rule 9), where they turn heuristic phishing detection into interpretation over verified channel facts — a brand claim from a sender that provably cannot be authenticated is a categorically stronger finding than a stylistic suspicion — while the layering stays one-directional: analysis can add warnings on top of the facts, never argue them away.

Result. The entire link-as-entry threat class is eliminated rather than mitigated: an operator can only ever enter a handshake through the trusted client, after independent cryptographic validation. Spoofed links, phishing invitations, and redirect chains do not become harder — they become irrelevant to handshake formation, because no link forms one (IX.3.1, IX.12.1). What remains for the attacker is distributing a code that resolves to the attacker's own, correctly verified identity — and that residue is exactly what the verification chain, attestation levels (P0/P1/P2), and the preview contract are built to make visible at the consent boundary, and what IX.12.4 scopes honestly: the system proves who the publisher is and what is executable, not that the publisher is virtuous.

IX.2 Definitions

Terms defined in Annexes II–VIII apply as defined there; in particular WR Code®, PoAE™ tiers, publisher attestation levels (P0/P1/P2), Manifest Root, grant, revocation, WCR recording, Presentation Surface. This Annex adds:

WR Execution Client. The governed runtime artifact specified by this Annex: a deployment form of the WR Desk™ runtime that forms Public Handshakes, maintains Handshake Contracts and Site Manifests, enforces the contract-bound rendering and execution rules, renders WR Views, and executes WCR recordings against them. The normative content of this Annex is engine-agnostic: it binds any conforming client regardless of the underlying rendering engine (IX.13).

Public Handshake. The public entry handshake of the web surface: formed exclusively by WR Code® scan or manual code entry (IX.3.1), binding the operator's runtime to a verified publisher and to that publisher's declared origin set, and yielding a Handshake Contract.

Handshake Contract. The long-lived relationship object produced by a Public Handshake (IX.4.1): publisher identity and attestation reference, publisher root key, bound origin set, protocol version, granted rights, and the Scope Envelope.

Hash-Pinned Consent. The consent discipline of this Annex: every consent event binds, by content hash, the exact material presented to the operator — the preview, the definition it was generated from, and the contract or manifest state it applies to — so that the consent record permanently and verifiably describes what was consented to (IX.3.4).

Assisted Code Capture. The capture method (IX.3.1) in which the client itself recognizes a WR code from inert, already-received data — a code recognized within the validated text of a depackaged BEAP™ message (assisted_email) or WR support detected via a Discovery Record on a visited or referenced domain (assisted_discovery) — runs the ordinary verification chain against it, and presents a Connect offer in client chrome. Capture is never formation: the recognized code is untrusted input exactly like a scanned or typed code, and a handshake forms only through Hash-Pinned Consent.

Channel Provenance Record. The typed, bounded verdict record produced automatically within the BEAP™ depackaging transformation for every message it processes and carried as typed fields of the resulting BEAP™ message (IX.3.1, rules 7–8): the evaluated channel-authentication results (e.g. SPF, DKIM, DMARC, presence and consistency of a Discovery Record for the authenticated sender domain) together with, where a WR code is present, the sender/publisher alignment finding. It is retained with the depackaged message, surfaced with identical semantics and identical alert presentation at every channel-consequential decision point — the Connect offer, the consent preview, and the external-link risk dialog — and provided as a declared input to the local scam analysis (IX.3.1, rule 9). It informs the operator's decision and is governed by the ratchet discipline: it can only ever increase friction, never confer or upgrade trust.

Fraud Watchdog™. The designation of the WR Desk™ local scam-analysis mode (IX.3.1, rule 9): a special analysis mode whose purpose is to make fraud, phishing, and social-engineering attempts locally visible through analysis. It runs as an auto-run analysis over validated message text, sender metadata, link strings, and the Channel Provenance Record, on host and sandbox; it never touches original artifacts, opens no attachments, follows no links, and transmits nothing off-device. Its findings are heuristic and advisory under the ratchet discipline: they make risk visible to the

operator and never carry, upgrade, or substitute for verification or authority. Fraud Watchdog™ is used as a product designation of the Optirando™ system.

Assessment Record Store. The append-only, publisher-facing record store on Optirando™ infrastructure holding attestation and verification records referenced during publisher verification (IX.3.5). Verification mechanisms write typed records against the store; adding a verification mechanism is a new module writing new record types, never a schema migration. The store holds attestation references, not operator data.

Scope Envelope. The outer boundary, declared in the Handshake Contract, of everything any Site Manifest version under that contract may declare: maximum data-access classes, maximum action categories, and the origin ceiling. Consent attaches to the Scope Envelope and to individual automations — never to content (IX.5.1).

Site Manifest. The versioned content object (IX.4.2), signed under the publisher's Manifest Root: page structure, resource hashes, declared dynamic-value slots, WR Link declarations, and the automation set with WCR recording hashes; monotonically versioned and freshness-bound.

WR View. The contract-bound rendered representation of a publisher's web presence: the deliberately reduced form of the site containing exclusively content declared in the active Site Manifest, rendered under strict passivity from the Verified Local Store. The WR View is a Presentation Surface (Annex V). It is a representation, not the original site: not the full page with ordinary browser capabilities, but the minimal version sufficient for the declared purpose.

Verified Local Store. The client-local content store from which WR Views are rendered exclusively: populated out-of-band by the client, every entry hash-verified against the active Site Manifest before admission (IX.6.2).

Pinned Rendering Subset. The formally defined subset of declarative markup and styling a WR View may contain (IX.6.1). Its exact grammar is an open specification item (IX.15); its exclusions are normative now.

Pinned Digest Algorithm. SHA-256: the digest algorithm over which every content hash of this Annex is computed (IX.4.4). Its change is a governance change, never a per-publisher or per-deployment option.

WR Trusted Execution Domain. The client-enforced boundary within which contract-bound rendering and execution occur: the Verified Local Store, the rendering context under render-time network starvation (IX.6.2), the slot-binding mechanism (IX.6.4), and the execution of WCR recordings under the contract (IX.8). Content enters the domain exclusively through hash verification against the active Site Manifest; dynamic execution paths are eliminated within it (IX.6.7). From v2.0, the term is generalized beyond the web surface per IX.16.2.

WR Compatibility Path. A publisher-authored alternative representation of a web presence, designed directly for the Pinned Rendering Subset and declared, normalized, hashed, and governed identically to any other manifest content (IX.6.8). It exists for sites whose ordinary representation does not remain functional under the transformation of IX.6; it receives no relaxation of any rule of this Annex.

WR Link. A declared, consent-gated trigger embedded in a WR View that invokes a specific, already-declared function or automation within an existing Public Handshake (IX.7). A WR Link is not an entry point and never establishes a handshake.

Invitation Class. The redemption semantics of a distributed WR code: public_bearer (open redemption; identity locks at formation) or targeted_bound (reserved; pre-formation recipient binding) — IX.3.2.

Discovery Record. A publisher-published DNS record enabling the client to detect WR support for a domain and to cross-check the publisher's commitment (IX.3.5). Discovery is never authority.

Change Class. The classification of a Site Manifest change as silent, notice, or consent (IX.5.3), determining the required operator interaction.

Manifest Version Pin. The declaration, by a WCR recording, of the Site Manifest version range against which it was validated (IX.8.1).

Finalizer. A deterministic enforcement point, declared in the Site Manifest and verified under the publisher root, through which every consequential, externally observable state transition of an automation passes (IX.8.4). The Finalizer verifies the requested effect against the agreed contract; it neither knows nor needs to know which code produced the request.

Decision Space. A manifest-declared selection space for the Selection execution class (IX.8.3): an enumerated option set and/or typed, bounded parameter ranges, together with the declared data sources whose runtime values may inform selection. Runtime data informs selection and never authorizes it.

Decision-Space Consent. The standing consent mode in which the operator consents to a declared Decision Space as such (IX.8.5): the client-generated preview presents the space — options, bounds, informing sources, selection rule, bound Finalizers — rather than a single instance.

Execution Capability. The unforgeable, attenuated authorization minted by a consent event (IX.8.5): bound to exactly the consented action or Decision Space, sender-constrained to the client core, carrying nonce, expiry, and idempotency binding. Finalizers accept proposals only under a valid Execution Capability.

Action Proposal. The structured, schema-constrained description of a single intended consequential effect, composed upstream and presented to validation, plausibility, consent, and the Finalizer (IX.8.3–IX.8.4). A proposal carries its expected-state binding and its target Finalizer version.

Plausibility Verification. The optional, staged check — rule-based, history-based, and at most advisorily model-based — interposed between validation and consent and independently at the Finalizer, governed by the ratchet rule: it may only ever increase friction, never reduce it (IX.8.7).

The execution-model terms introduced in v2.0 — the generalized WR Trusted Execution Domain, Intent Hash, Transition Class, Consent Mechanism Class, Verification Tier, and Transaction Envelope — are defined in IX.16.2.

IX.3 The Initiation Layer

The initiation layer governs how Public Handshakes come into existence. It is surface-independent: the web surface of this Annex is its primary instantiation, and Annex VII cites this layer for

handshake initiation generally. Nothing in this section creates execution authority; initiation yields delivery and preparation rights only (Annex VII §10).

IX.3.1 Entry Points, Capture Methods, and Code Distribution (Normative)

A Public Handshake SHALL be formed exclusively from a WR code obtained through one of three capture methods:

1. scanning a WR Code® with the client (client-controlled scan path, fail-closed);
2. manual entry of a WR code into the client;
3. **Assisted Code Capture** — client-side recognition of a WR code from inert, already-received data, per the rules below.

A capture method obtains the code; it forms nothing. Whatever the capture method, the captured code is untrusted input, and formation follows one identical path: the verification chain of IX.3.4/IX.3.5, the client-generated preview of the verified publisher under the preview contract of IX.5.3, and Hash-Pinned Consent. No other mechanism establishes a Public Handshake. In particular: a WR Link SHALL NOT establish, extend, or upgrade a handshake (IX.7.1); a Discovery Record SHALL NOT establish a handshake — it MAY at most trigger Assisted Code Capture (IX.3.5); an ordinary hyperlink, deep link, push notification, or NFC/QR payload outside the WR Code® scheme SHALL NOT establish a handshake. Capture initiates; it does not complete. The code confers no authority of its own; capture — including assisted capture — confers no authority; only consent completes entry.

Assisted Code Capture. The client MAY recognize a WR code without operator transcription, from exactly two source classes:

- **assisted_email** — a WR code recognized within a depackaged message. Depackaging transforms the received email into the BEAP™ format: machine- and human-readable, text-pure under allowlist positive construction, secure and deterministic by design. The original artifact never crosses the boundary (blob-inertness: originals remain encrypted in the depackaging context); what the client holds is the validated BEAP™ representation, and the WR code is recognized within that validated text. Only the code token parameterizes the capture flow.
- **assisted_discovery** — WR support detected for a visited or referenced domain via its Discovery Record (IX.3.5).

Assisted Code Capture is governed by the following rules:

1. **Offer, never action.** Recognition yields at most a Connect offer: the client runs the full verification chain of IX.3.4/IX.3.5 and, on success, MAY present in client chrome the verified publisher identity, attestation level, handshake scope, and capture provenance. Presentation of the offer confers no authority (the preview rule of IX.5.2 applies); the offer is not a navigation, not a formation, and not a granted right.
2. **Untrusted input, unconditional verification.** A recognized code is untrusted input identical to a scanned or typed code. Verification is unconditional and fail-closed; a failed verification SHALL suppress the Connect offer entirely — no “connect anyway” path exists.
3. **Instruction-incapable parameterization.** The capture flow is parameterized exclusively by the WR code’s typed, bounded identifier grammar — the admission discipline of IX.8.6

applied to capture. The validated BEAP™ text within which the code was recognized is ordinary safe message content under the BEAP™ text-purity discipline; it, and any link it carries, SHALL NOT trigger, parameterize, or shortcut the flow, and the original message artifact never crosses the boundary at all.

4. **Channel passivity.** No navigation event, link activation, push payload, or any other channel-delivered mechanism SHALL trigger, parameterize, or shortcut Assisted Code Capture. Detection is performed by the client, on inert data the client already holds or on the passive Discovery Record; the channel remains untrusted per this section throughout.
5. **Provenance.** The capture provenance (assisted_email or assisted_discovery, with source reference) SHALL be displayed in the consent preview and recorded in the Handshake Contract (IX.4.1) and the evidence log (IX.11).
6. **Policy governance.** Under a Secure Browse policy (IX.9), Assisted Code Capture MAY be disabled entirely or downgraded to notice-then-manual-confirmation. Scan and manual entry are never removable and are never replaced by assisted capture; manual entry retains its high-assurance role per IX.1.5.
7. **Channel Provenance Verification.** Where the capture source carries verifiable channel metadata — for assisted_email, the message's channel-authentication state — the channel standards SHALL be evaluated automatically within the BEAP™ depackaging transformation itself (SPF, DKIM, DMARC, and the presence and consistency of a WR Discovery Record for the authenticated sender domain, per deployment configuration), and the resulting **Channel Provenance Record** SHALL be carried as typed, bounded fields of the depackaged BEAP™ message — one transformation, verdicts attached to its output by design. Raw headers and signatures remain with the original artifact and do not cross. The Connect offer and the consent preview SHALL state the verdicts as facts — including whether the authenticated sender domain aligns with the verified publisher's bound origin set — so that the operator can make a fact-based decision whether a Public Handshake should be formed from this channel at all. Channel verdicts are governed by the ratchet discipline (IX.8.7 applied to capture): they inform, and they MAY only ever increase friction — a failing, absent, or misaligned verdict MAY suppress the offer, downgrade it to notice-then-manual-confirmation, or place a stated finding in the preview. They SHALL NOT upgrade trust, reduce any friction, pre-satisfy any consent, or substitute for any step of the WR verification chain, whose validity is independent of the channel. **Claims discipline (IX.0.1):** channel standards authenticate the sending domain — not the publisher's intent and not the code's safety; the structural guarantee of this Annex, that even a hostile publisher can effect nothing beyond the verified contract, does not depend on them, and a fully passing channel is not evidence of a trustworthy publisher. The Channel Provenance Record is retained in the evidence log (IX.11).
8. **Uniform surfacing of channel verdicts.** Channel Provenance Verification is not specific to codes: the BEAP™ depackaging transformation SHALL evaluate the channel standards of rule 7 for every message it processes, and the Channel Provenance Record travels with the resulting BEAP™ message wherever it is consumed. The record SHALL be surfaced, with identical verdict semantics and identical alert presentation, at every channel-consequential decision point — at minimum: (a) the Connect offer and the consent preview (rule 7); and (b) the external-link risk dialog — the dialog the client presents before any external link

carried in a depackaged message is opened. Where DKIM and DMARC are absent or unverifiable for the message, both surfaces SHALL display a prominently highlighted warning of the alert class (red-highlighted, identical styling on both surfaces), stating that the authenticity of the sender address could not be verified because the sending domain publishes no verifiable channel authentication — and that links and codes carried in the message therefore have no verified origin. The warning states unverifiability, not maliciousness: absence of channel authentication is a fact about the channel, not a verdict about the sender, and the claims discipline of rule 7 applies unchanged. The warning is ratchet-conforming: it adds a stated finding and SHALL NOT be suppressible by message content, publisher content, or any non-policy mechanism; a Secure Browse policy (IX.9) MAY escalate it — up to blocking external-link opening from unauthenticated channels — but never remove it. One evaluation at depackaging, one record, one presentation, every decision point.

9. **Fraud Watchdog™ integration.** Where the deployment operates the local scam-analysis mode Fraud Watchdog™ (IX.2) — the auto-run analysis that makes fraud, phishing, and social-engineering attempts locally visible by examining validated message text, sender metadata, and link strings, on host and sandbox, without touching original artifacts, opening attachments, or following links — the Channel Provenance Record SHALL be provided to it as a declared, typed input, and the analysis SHALL incorporate the verdicts into its phishing-risk assessment. The deterministic channel facts strengthen the analysis's cross-checks by construction — in particular the brand-impersonation check: a brand claim plus an action request plus an unauthenticated or publisher-unaligned sender is a materially stronger finding than either signal alone. The layering between the two mechanisms is strict and one-directional: rule-7/8 verdicts are facts — deterministic, channel-derived, presented unconditionally; Fraud Watchdog™ is interpretation over facts — heuristic, model-based, advisory in the sense of the ratchet discipline (IX.8.7(1)(c) applied to message analysis). Fraud Watchdog™ findings MAY be surfaced at the decision points of rule 8 alongside the deterministic verdicts, visually distinguished as analysis findings rather than verified facts, and MAY escalate presentation or add stated findings; they SHALL NOT suppress, soften, precede, or replace the deterministic warning of rule 8, SHALL NOT upgrade trust or reduce friction on any surface, and a benign Watchdog assessment of an unauthenticated channel does not remove the unverifiable-sender warning. Analysis runs locally; neither the record nor the analyzed content is transmitted off-device for this purpose, consistent with the local-evidence discipline of IX.11. **Claims discipline (IX.0.1):** Fraud Watchdog™ is a heuristic, false-positive-averse detector; its findings are advisory and are not represented as verification. Its operational safety as a model-based stage is an objective in the sense of IX.12.2.

Code distribution channels. A WR code MAY be distributed over any channel — printed matter, on-screen display, or invitation messages such as email. Distribution is not entry: an emailed invitation SHALL carry only a human-readable WR code and a short explanatory text. Binary capsules, executable attachments, and formation-triggering links SHALL NOT be carried in invitation messages; clicking any link in an invitation SHALL NOT form a handshake. Entry occurs when the recipient enters the code into the client (entry point 2), whereupon the full verification chain runs client-side. The invitation channel is therefore untrusted by construction: nothing it carries is executed, and nothing it carries confers authority.

The code is an identifier and invitation — not authority. Authority arises only from successful client-side cryptographic verification. A distributed code — printed, displayed, or messaged — is untrusted input in every form, including when scanned: it becomes the basis of a handshake only through the verification chain of IX.3.4/IX.3.5 and Hash-Pinned Consent. A hyperlink accompanying an invitation MAY carry explanatory information or assist the operator in locating the official client; it SHALL NOT initiate, deep-link into, pre-populate, or otherwise shortcut the handshake flow. This prohibition binds the channel, not the client: Assisted Code Capture — the trusted client recognizing a code within inert, already-received data under the rules above — is not a channel-initiated flow and does not fall under it. Distribution is not entry, and detection is not entry either: consent completes entry. The rationale for this rule — the link-as-entry failure class — is given in IX.1.5.

IX.3.2 Invitation Classes (Normative)

Two invitation classes are defined:

- **public_bearer** — the open-redemption class: the code is redeemable by whoever holds it. This is the deliberate semantics of public entry points (posters, packaging, websites, broadcast invitations); pre-formation recipient-binding is not a property of this class and SHALL NOT be imposed on it. All public web-surface handshakes of this Annex are public_bearer.
- **targeted_bound** — reserved canonical name for targeted invitation flows with pre-formation recipient binding. Semantics are deferred to a future revision; the name is registered to prevent divergent usage.

IX.3.3 Formation, Single Consent, and Identity Lock (Normative)

Single consent. Where a WR code prepares both a handshake and a first declared automation, formation MAY proceed under a single Hash-Pinned Consent covering exactly both — the handshake (with its Scope Envelope) and the identified first automation — presented together under the preview contract (IX.5.3). The single consent is a usability rule, not an authority rule: it covers precisely what its hashed preview presented, and nothing further.

Identity lock at formation. Upon formation, the stable identity fingerprints of both endpoints (SSO subject identifier and realm, per the established identity-binding model) SHALL be bound into the Handshake Contract. From formation onward the relationship is identity-bound; bearer semantics apply only up to formation. Post-formation re-verification of a typed address or similar out-of-band identity confirmation is redundant by construction and SHALL NOT be required.

Standing relationship. A formed Public Handshake is a persistent standing channel under the Annex VII grant model: delivery and preparation rights only, no-expiry-until-revoke as the ground state, unilaterally and silently revocable by the operator at any time.

IX.3.4 Handshake Invariants and Origin Binding (Normative)

The Public Handshake carries three invariants, normative in this Annex:

1. **Hash-Pinned Consent.** Every consent event in handshake formation and thereafter binds, by content hash, exactly what was presented: the preview, the bound definition it was

generated from, and the contract state it applies to. A consent record that cannot be resolved to the hashed material it authorized is invalid.

2. **The preview contract.** Every consent-gated presentation is generated by the client from the cryptographically bound definition — never from publisher free text — per IX.5.3.
3. **Origin binding (web-surface specific).** The Handshake Contract binds an explicit origin set (scheme, host, port — enumerated, no wildcards across registrable domains), and every Site Manifest, resource, WR Link target, and automation under the contract SHALL resolve within that set.

Handshake mechanics not specific to the web surface — profile structure, capability declaration, the grant model, revocation — follow Annex VII unchanged.

IX.3.5 Publisher Verification and Discovery (Normative)

Publisher verification follows the two-entity attestation model of Annex VII (P0/P1/P2) with references resolved against the Assessment Record Store (IX.2). For the web surface, verification is dual-channel:

1. **DNS channel.** The publisher publishes a Discovery Record on the domain: a commitment to the publisher root key fingerprint and/or the current Manifest Root hash, plus the WR protocol version. Because DNSSEC deployment cannot be assumed, the DNS channel alone SHALL NOT be treated as authoritative.
2. **Origin channel.** A well-known endpoint on the bound origin, served over TLS with WebPKI validation, serves the current signed Site Manifest (or a pointer to it) committing to the same root.

Both channels SHALL agree with each other and with the attestation reference in the Assessment store; disagreement is a hard verification failure (fail-closed, no handshake). Adding further verification modules (e.g. an operator-facing data-verification process) is, by the Assessment store contract, a new module writing new record types against the same append-only store — never a schema migration.

Discovery is never authority. The client MAY use Discovery Records to indicate WR support for a visited or referenced domain (“this publisher supports WR”), to pre-fetch verification material, and to trigger Assisted Code Capture (assisted_discovery, IX.3.1); it SHALL NOT auto-form a handshake, pre-grant any right, or weaken the entry rule of IX.3.1 on that basis. Presenting a verified Connect offer under Assisted Code Capture is not auto-formation: the offer exists only after the full verification chain has succeeded, confers no right, and forms nothing without Hash-Pinned Consent.

IX.3.6 Transport-Privacy Tiers (Normative)

Transport under a public_bearer handshake is tiered:

- **Default (Beta):** the sealed coordination relay — complete sealed BEAP™ capsules over the blind-courier relay, per the ratified relay transport model. The relay observes routing metadata only, never plaintext.

- **Disclosed opt-in:** direct P2P transport MAY be offered; because P2P exposes endpoint network metadata to the counterparty, it SHALL be presented as an explicit, disclosed choice, never a silent default.
- **High-Assurance tier (reserved):** split-hop relaying and transport-level Obfuscation Mode are reserved for the High-Assurance tier and are not claimed for the default path.

Tier selection is local policy plus operator choice; a publisher SHALL NOT be able to force a less private tier than the operator's policy permits.

IX.3.7 Escalation to Full-Context Classes (Normative)

A Public Handshake never escalates silently. Upgrading a public_bearer relationship to a full-context, bidirectional handshake class (the org-class profiles of Annex VII) is a new consent event under the ordinary Annex VII profile formation rules, with its own preview and Hash-Pinned Consent. Nothing accumulated under the public relationship — history, delivered content, executed automations — pre-satisfies any step of the escalation (Annex II §II.2.5).

IX.3.8 Reserved: Credential Attachment Envelope

The name **Credential Attachment envelope** is reserved for the initiation-layer mechanism carrying operator credentials into declared flows. Its normative semantics are deferred to a future revision of this Annex; until then, no conforming implementation SHALL ship a mechanism under this name.

IX.4 The Two-Object Contract Model

The contract between operator and publisher consists of exactly two signed objects under one publisher root. Rationale (informative): a single object embedding content in the relationship would force every content update to be either a re-consent event (consent fatigue destroys the signal) or a silent mutation of a consented object (the consent record no longer describes what runs). Two objects keep the consent record permanently accurate: the operator consented to an envelope and to specific automations; content moves provably inside that.

IX.4.1 The Handshake Contract (Normative)

The Handshake Contract is the long-lived relationship object. It SHALL contain at minimum: publisher identity and attestation level with Assessment store reference; the publisher root key; the bound origin set; the protocol version; the invitation class and recorded capture method (code scan, manual entry, assisted_email, or assisted_discovery — IX.3.1), including capture provenance for assisted methods; the granted rights — delivery and preparation rights only, per the Annex VII grant model, with no-expiry-until-revoke as the ground state and unilateral silent revocation by the operator at any time; and the Scope Envelope. The Handshake Contract SHALL change only through a consent event (IX.5.3, class consent). Publisher root key rotation is a new trust decision distinct from revocation and SHALL be treated as a consent event; it SHALL NOT be processed as a silent update under any circumstances.

IX.4.2 The Site Manifest (Normative)

The Site Manifest is the versioned content object, signed under the publisher's Manifest Root using the Annex VII artifact machinery unchanged. It SHALL declare: the page set and structure within

the Pinned Rendering Subset; the complete resource set as content hashes of normalized derivatives (IX.6.3); the declared dynamic-value slots with types and bounds (IX.6.4); WR Link declarations (IX.7); and the automation set, each automation identified by its WCR recording hash and its declared action, read, write, and transfer sets. Every content hash the Site Manifest declares — resource hashes, WCR recording hashes, and the hash bindings of WR Link targets — is a SHA-256 digest per IX.4.4.

Rendering is governed by the approved Site Manifest, never by the original website. The website is the source of manifest content during preparation and normalization; it holds no authority over rendering. Once a manifest version is verified and active, subsequent changes to the originating site — whether ordinary evolution or compromise — have no effect on what the client renders until a new manifest version passes the verification chain and, where its change class requires it, the consent boundary of IX.5.

Versioning and freshness (anti-rollback). Site Manifest versions SHALL be strictly monotonic under a per-contract counter. The client SHALL reject any manifest whose version is lower than the highest version already accepted under the contract, regardless of signature validity — a validly signed stale manifest is the rollback attack, not a valid state. Manifests SHALL carry a signed freshness assertion; where freshness cannot be established (offline, endpoint unreachable), the client SHALL fail closed for new content while remaining entitled to render the last verified state it already holds, visibly marked as potentially stale, with all automations suspended until freshness is re-established.

IX.4.3 Signing, Revocation, and Custody (Normative)

Both objects verify against the publisher root through the Annex VII verification chain (§7). Revocation follows Annex VII semantics: operator-side revocation of the Handshake Contract is unilateral, silent, and immediate; publisher-side withdrawal of a manifest version or automation takes effect at the client upon the next freshness check and SHALL suspend affected recordings (IX.8.1). Handshake keys and consent-capture SHALL reside in the client core, backed by platform hardware key storage where available, and SHALL NOT be accessible to any rendering context (IX.10).

IX.4.4 Integrity Verification and the Pinned Digest Algorithm (Normative)

SHA-256 is the **Pinned Digest Algorithm** of this Annex — a governance-level constant of the verification chain, in the same sense in which WebP is the pinned raster serialization (Annex VIII) and safetensors the pinned adapter serialization (Annex IV §IV.3.2). A change of the pinned digest algorithm is a governance change of the Annex VII artifact machinery, never a per-publisher or per-deployment option. This pinning is governance-level cryptographic agility, not permanence: hash and signature algorithms are replaceable through the ordinary governance path without any change to the higher-level trust architecture, and hash-based integrity is emphasized deliberately for its long-term robustness, including under post-quantum assumptions.

The integrity discipline is uniform across everything this Annex declares:

1. **Digest generation.** Every approved WCR recording generates a SHA-256 digest over its canonical serialization at approval; every normalized resource derivative generates a SHA-256 digest at normalization (IX.6.3); every WR Link target and bound definition is

identified by SHA-256 digest. These digests become part of the Site Manifest and verify against the publisher root through the Annex VII verification chain.

2. **Verification before processing.** WCR recordings and WR View content are protected through SHA-256 based integrity verification: during replay and during rendering, every referenced resource and every recording is verified against the approved manifest before processing (IX.6.2 for resources, IX.6.3 hash-before-decode for media, IX.8.1 for recordings). Verification is unconditional on every render and replay path; there is no trusted-because-cached, trusted-because-local, or trusted-because-recently-verified shortcut.
3. **Rejection semantics.** A digest mismatch is a deviation under IX.6.5: the artifact is rejected — never repaired, partially rendered, or substituted — and modified recordings do not execute. WCR recordings and manifest-declared content are thereby cryptographically verifiable artifacts: their exact approved content is demonstrable at every use, and any deviation from the approved state is detected and refused rather than processed.
4. **Claims discipline (per IX.0.1).** This construction is not represented as making recordings "tamper-proof": no hash function does. What it provides is that modifications become cryptographically detectable — producing an altered artifact that verifies against an approved digest requires defeating the second-preimage resistance of SHA-256, computationally infeasible under current cryptographic assumptions — and that integrity is enforced through manifest verification, i.e. through the unconditional pre-processing check of clause 2, not through any property of stored bytes or storage location.

IX.5 Change Classes and the Consent Boundary

IX.5.1 Generating Principle (Normative)

Consent attaches to the Scope Envelope and to individual automations — never to content. A change requires operator interaction exactly to the degree that it touches what the operator consented to: the envelope, or a consented automation's declared behavior. Content that moves within the envelope, in the passive subset, with no new action or data access, is the publisher's to change.

IX.5.2 Change Classes (Normative)

Silent update — no operator interaction. (a) New values bound at runtime into already-declared slots (prices, availability, account state) — never free markup injection (IX.6.4); (b) resource replacements that are re-normalized, re-hashed, and re-signed under a manifest version bump; (c) page additions, removals, or restructurings within the bound origin set that declare no new action, no new data access, and remain within the Pinned Rendering Subset; (d) textual and layout changes within the subset. The client verifies the new version against the root on next use; no interaction is required. **Tier split:** under a Secure Browse policy (IX.9), class (c) — structural growth — SHALL be promoted to notice.

Notice — visible, non-blocking. Changes the operator should see but that expand nothing: relocation or addition of WR Links referencing already-consented automations; regeneration of an existing automation's preview because its bound definition was re-signed at identical scope; manifest version changes under a Secure Browse policy per the tier split above. Notices SHALL be

surfaced in client chrome (never inside the WR View content area, which the publisher controls) and logged; they SHALL NOT require a consent tap.

Consent — blocking, PoAE™-recorded. (a) Any new automation; (b) any change to an existing WCR recording's action set, reads, writes, or transfer targets; (c) any new origin; (d) any expansion of the Scope Envelope; (e) publisher root key rotation; (f) downgrade of the publisher's attestation level. Each SHALL re-run the preview contract of IX.5.3: the preview is generated by the client from the cryptographically bound definition, never from publisher free text (this rule is the preview contract), and SHALL state what executes, what is read, what is transmitted, what is changed, what persists, where further confirmation will be required, and what is expressly not executed. Presentation of a preview confers no authority; execution occurs only on the human consent tap with a PoAE™ record (VII-10.1).

IX.6 The WR View — Rendering and Enforcement

IX.6.1 Strict Passivity and the Pinned Rendering Subset (Normative)

A WR View SHALL contain no active content. Excluded categorically: JavaScript, WebAssembly, service and shared workers, plugin content, undeclared iframes, dynamically loaded active content, timers and scheduled callbacks, asynchronous execution interfaces of the host engine (event-driven and deferred-execution APIs through which content could act after render), and any scripting or code-execution capability of the host engine, which SHALL be disabled at the engine boundary where the engine offers such control. The WR View is composed exclusively within the Pinned Rendering Subset: a declarative markup and styling subset whose exact grammar is an open specification item (IX.15) but whose exclusions above are normative immediately. The rendering discipline is that of Annex VIII's rendering target — strict passivity, deterministic feature subset, exact self-containment — applied to the web surface.

IX.6.2 The Verified Local Store and Render-Time Network Starvation (Normative)

The client SHALL fetch declared content out-of-band, verify every artifact's hash against the active Site Manifest before admission, and admit only matching artifacts into the Verified Local Store. WR Views SHALL be rendered exclusively from that store. At render time the rendering context SHALL have no network path: every resource request originating from the rendering context SHALL be intercepted and answered from the store or not at all. An undeclared resource is therefore not "blocked" as a policy action — it has no route to the network by construction; the failed resolution SHALL trigger the deviation rule of IX.6.5. Navigations, redirects, downloads, external protocol invocations, and pop-ups SHALL pass exclusively through client-controlled interception and are permitted only where declared in the manifest; all else fails closed.

IX.6.3 Media Normalization (Normative)

Media in a WR View are normalized derivatives, never publisher originals. Raster images SHALL be normalized upstream on Optirando™ infrastructure — decoded, stripped of metadata and non-essential structure, re-encoded to WebP as the pinned raster serialization (Annex VIII), hashed, and declared in the Site Manifest. The client SHALL compare the hash of the loaded artifact against the manifest-declared hash before decode; only a matching artifact is decoded and rendered. The

derivative is visually and informationally equivalent to the source, not byte-identical to it; the original is not rendered in a WR View. The normalization instance is a security-critical supply-chain component and SHALL itself be isolated, versioned, tested, and auditable; its converter is tool-governed with pinned identity and content hash, consistent with the converter governance of Annex V. Other media classes SHALL follow the same normalize-hash-verify pattern before any WR View rendering; their format-specific treatment is deferred (IX.14). **Video is excluded from WR Views** in this version and is a roadmap item (IX.14); a conforming v1 client SHALL NOT render video in a WR View. Where normalization and the passive subset leave a publisher's ordinary representation non-functional in WR form, the publisher-side resolution is the WR Compatibility Path (IX.6.8) — never a relaxation of this section.

IX.6.4 Dynamic Values via Declared Slots (Normative)

Live values (prices, stock, account state) enter a WR View exclusively through slots declared in the Site Manifest with type, format, and bounds. Slot binding is pure value binding: validated against the declaration, rendered as inert content, never interpreted as markup. Free HTML injection into a WR View does not exist as a mechanism. This is the slot discipline of Annex VIII (VIII.4.4) applied to live web content, and it is the normative answer to dynamic content: static structure is hash-bound per manifest version; dynamic values are typed, bounded, and bound at runtime.

IX.6.5 Deviation and Hard Stop (Normative)

On any relevant deviation — hash mismatch, undeclared resource resolution attempt, slot-bound value violating its declaration, navigation outside the declared set, manifest freshness failure during an active session, or state divergence during automation (IX.8) — the client SHALL: detect, block, safely stop the affected session or operation, inform the operator intelligibly in client chrome, and record the event. Silent continuation past a deviation is non-conforming. Degradation of non-deviating content follows Annex II §II.2.8.

IX.6.6 Presentation Integrity (Normative)

The presentation integrity rules of Annex VIII §VIII.7 apply to WR Views: no element displays before validation; late-arriving content fills reserved geometry without layout shift; reveal is atomic per validated region.

IX.6.7 The Deterministic Execution Model (Normative)

The mechanisms of IX.6.1–IX.6.4 compose into a deliberate architectural transformation: dynamic web content is converted into a deterministic execution model, enforced within the WR Trusted Execution Domain. Inside the domain:

- JavaScript execution is disabled — categorically in the Pinned Rendering Subset, and at the engine boundary where the engine offers such control (IX.6.1);
- timers and scheduled callbacks are disabled (IX.6.1);
- asynchronous browser APIs are disabled (IX.6.1);
- unauthorized network communication is blocked — more precisely, it has no path by construction (IX.6.2);

- dynamic execution paths are eliminated: no mechanism exists through which content could act, load, or transform after render;
- only previously approved resources — manifest-declared and hash-verified per IX.4.4 — are processed.

The resulting security property is stated in two parts, corresponding to the two mechanisms that produce it. First: because every capability through which post-render change could be effected is excluded from the domain, the rendered WR View cannot evolve beyond the previously approved manifest state — no execution path exists through which the presented content could change after verification, dynamic values entering exclusively through declared, typed, bounded slots (IX.6.4). Second, and independently of any policy profile: resources outside the approved Site Manifest are never processed (IX.6.2, IX.4.4); even where a future profile permits additional capability, the material available within the domain remains confined to the approved, hash-verified set.

This transformation significantly reduces attack surface, structurally rather than reactively: vulnerability classes that presuppose dynamic execution — script injection, time-of-check/time-of-use divergence between verification and rendering, post-load content substitution, exfiltration through scripted network access — are not mitigated within the domain but made inexpressible in it. It simultaneously improves reproducibility and execution determinism: the rendered state is a function of the approved manifest version, the verified resource set, the bound slot values, and the client's pinned subset — with determinism understood as in IX.8.2, a deterministically bounded state and action set with verifiable inputs, not bit-identical rendering across devices — and it is this bounded determinism that makes the auditability of IX.11 achievable: what was shown is demonstrable because what could be shown was closed in advance.

IX.6.8 The WR Compatibility Path (Normative)

Not every web presence survives the transformation of IX.6.1–IX.6.4 functionally intact: a site whose essential function depends on active content, on undeclared dynamism, or on media classes outside the pinned set may render correctly as a reduced representation and yet cease to function as intended. The resolution is never a relaxation of the subset, of normalization, or of any rule of this Annex; it is a second authoring path on the publisher's side:

1. **Dedicated representation.** A publisher MAY maintain, within its own codebase, a representation authored specifically for the WR View — the WR Compatibility Path: an alternative page set targeting the Pinned Rendering Subset directly, with declared slots for its dynamic values (IX.6.4), declared WR Links for its interactive functions (IX.7), and declared automations for its flows (IX.8). Where the ordinary representation degrades or breaks under transformation, the client renders the Compatibility Path representation in its place.
2. **No special case.** A Compatibility Path representation traverses the identical pipeline — upstream normalization (IX.6.3, including the replacement of media elements by normalized derivatives that are visually and informationally equivalent but new files), SHA-256 hashing (IX.4.4), Site Manifest declaration and versioning (IX.4.2), the verification chain, the change classes (IX.5), and render-time network starvation (IX.6.2) — and receives no relaxation of any rule. Its only distinction is authorship: it is designed for the subset rather than reduced into it, which is why it functions where a reduction would not.

3. **Equivalence responsibility.** Functional and informational equivalence between a publisher's ordinary web presence and its Compatibility Path representation is the publisher's responsibility and lies outside the trust model, consistent with IX.12.4: the system proves that the declared representation is exactly what renders and exactly what executes; it does not and cannot prove that the representation faithfully mirrors the publisher's ordinary site.
4. **Specification obligation.** WR Desk™ SHALL publish the declaration format and authoring specification for the Compatibility Path, so that publishers whose sites display incorrectly in the WR system have a defined, tool-supported way to offer a conforming representation. The formal declaration format, authoring rules, and validation tooling are open specification items (IX.15).

Informative. The closest established precedent is the publisher-authored parallel representation in a validated restricted format (IX.14.2); the Compatibility Path differs in being hash-bound, manifest-declared, change-class-governed, and rendered under render-time network starvation. The pattern also completes the incentive structure: the strictness of the WR View never needs to bend to accommodate a site, because the accommodation mechanism lives on the authoring side, under full governance.

IX.7 WR Links

IX.7.1 Position (Normative)

A WR Link is a secondary, consent-gated trigger inside an existing Public Handshake. It SHALL be declared in the Site Manifest, SHALL reference a declared function or automation by hash, and SHALL be rendered only within a WR View of a contract with a valid handshake. A WR Link SHALL NOT establish a handshake, SHALL NOT be actionable outside a valid handshake, and SHALL NOT reference anything not declared in the active manifest. WR Links MAY be offered anywhere in the declared page set — FAQ sections, help pages, account settings, product pages, service flows.

IX.7.2 Activation Flow (Normative)

1. A valid Public Handshake exists; the operator is in the corresponding WR View.
2. The operator activates a declared WR Link.
3. The client loads the bound definition referenced by the link (never a free-text description).
4. The client generates and displays the preview per the preview contract (IX.5.3).
5. The operator grants explicit consent (PoAE™ tier per the action's class; the tap is the execution authorization, VII-10.1).
6. Exactly the declared function executes; execution is monitored against its declaration.
7. Any relevant deviation triggers IX.6.5.

Displayed execution, technically permitted execution, and actually performed execution SHALL coincide; divergence between any two is a deviation.

IX.8 Automation — Execution Classes, WCR Recordings, and the Consequence Boundary

IX.8.1 Manifest Version Pinning (Normative)

Every WCR recording executed against a WR View SHALL carry a **Manifest Version Pin**: the Site Manifest version range against which it was validated. A recording SHALL bind, at minimum: publisher, origin set, Handshake Contract reference, protocol version, Manifest Version Pin, its declared action/read/write/transfer sets, expected states, resource hashes it depends on, applicable local policy, session parameters, and the consent reference. The recording itself is identified by its SHA-256 digest in the Site Manifest, and replay verifies the recording and every resource it depends on against the approved manifest before processing (IX.4.4). A manifest version bump that touches any resource, slot, page, or state the recording depends on **invalidates the pin**: the recording SHALL suspend fail-closed until the publisher re-signs it against the new version. Re-enable is silent where the recording's own definition is unchanged (the site changed around it) and a consent event where the definition changed (IX.5.3). "Latest version" is not a valid pin. This is the propose-never-silently-apply principle at the web surface: no automation silently adapts to a changed site.

IX.8.2 Execution Semantics (Normative)

Only declared steps execute. An unknown action, an unexpected state, a changed resource, or an out-of-declaration value terminates the run per IX.6.5, with the divergence recorded. Determinism here means a **deterministically bounded state and action set with verifiable inputs, resources, and transitions** — not bit-identical rendering across devices. Automation confers no authority: recordings run under delivery and preparation rights; every execution is authorized by its consent per the PoAE™ tiering, and no manifest content, link placement, or preview pre-satisfies any tier (Annex II §II.2.5).

IX.8.3 Execution Classes (Normative)

Automation under this Annex executes in exactly one of three classes, distinguished by where selection latitude resides. The class is declared in the Site Manifest per automation and is part of the consented definition.

1. **Replay Class.** The automation is a pinned WCR recording per IX.8.1–IX.8.2. Model participation is limited to path tolerance — absorbing non-semantic variance within declared tolerance — and never alters tool-boundary behavior: at every consequential step, the recorded, declared action executes, or the run terminates (IX.6.5). Replay is deterministic at the tool boundary and adaptive on path; it is recorded operation carried into semantic form and re-executed from it, with the model as tolerance compensation, never as decision-maker.
2. **Selection Class.** The Site Manifest declares a Decision Space (IX.2), and a model selects within it: it chooses among the enumerated options and binds parameters within the declared bounds, informed by the runtime values of the declared data sources. Each selectable option is, in itself, a declared automation in the ordinary sense of this Annex; selection determines which declared behavior executes, never what a behavior is. Runtime data informs and

never authorizes (the non-pre-satisfaction rule of Annex II §II.2.5 applied to runtime values: information without authority).

3. **Proposal Class.** A model composes an Action Proposal not covered by any standing Decision-Space Consent. A Proposal-class action SHALL always be individually consented (IX.5.2, class consent), previewed from its bound definition, and executed through the Finalizer like every other class.

Class invariance. Whatever the class, the consequence boundary is identical: validation, Plausibility Verification where configured (IX.8.7), consent at the applicable PoAE™ tier, the Finalizer (IX.8.4), receipt, evidence (IX.11). The security statement of this Annex is therefore independent of the intelligence of the path: greater upstream latitude shifts load onto the consequence boundary — it never degrades the guarantee, because the guarantee was never attached to the path.

Out-of-space divergence. Where runtime data falls outside its declared expectations, or where no option can be selected at the deployment's declared confidence, the divergence discipline applies: the condition is surfaced, the operation stops or escalates to the operator, and nothing is silently widened — an option SHALL NOT be executed below the declared threshold merely because it is the best available (the threshold discipline of Annex VIII §VIII.6.5.2 applied to actions).

Informative — the Annex VIII symmetry. Annex VIII mandates deterministic Branch Conditions for presentation because there, selection is itself the consequence: an incorrect artifact displayed with visual authority is the harm. Here, selection may be intelligent because Finalizer and consent stand between selection and consequence. It is the same philosophy with mirrored placement of the deterministic element.

IX.8.4 The Finalizer (Normative)

Every consequential, externally observable state transition of an automation — database commits, payment execution, configuration changes, message dispatch, any persistent or transmitted effect — SHALL pass through a Finalizer.

1. **Declared and verified before use.** A Finalizer is a manifest-declared artifact: identity, endpoint, effect schema, and version, committed under the publisher's Manifest Root and verified through the Annex VII chain (§7) before any Execution Capability is minted against it — the tool-governance discipline of pinned identity and content hash, applied to the enforcement point itself. Every Action Proposal binds the Finalizer version it targets; a version unknown to, or older than, the verified manifest state is rejected (the anti-rollback discipline of IX.4.2 applied to enforcement points). Otherwise a compromised path would not need to exchange the action — exchanging the instance that checks it would suffice.
2. **Effect verification, not code verification.** The Finalizer verifies the requested effect against the agreed contract: the proposal against the declared effect schema and bounds; the expected-state binding against current state, committed atomically only on match (compare-and-swap; on mismatch, abort — never silent continuation); the Execution Capability and consent evidence; idempotency, with the proposal hash as the idempotency key; and freshness (nonce, expiry). It does not, and need not, verify which code produced the request — that is the point of Effect Binding (IX.8.8).

3. **Completeness.** No path to an externally observable consequence SHALL exist except through a declared Finalizer. Reads and transmissions are consequences in this sense: within the web surface this is already structural — the rendering context has no network path (IX.6.2), and every read and transfer target of an automation is declared and consent-covered (IX.8.1) — and a conforming implementation SHALL NOT introduce any effect channel that bypasses the Finalizer discipline. A verified Finalizer is necessary; it becomes sufficient only together with this completeness rule.
4. **Receipt.** On commit, the Finalizer returns a signed execution receipt — proposal hash, resulting-state binding, timestamp, Finalizer version — which the client verifies and retains in the evidence chain (IX.11). The receipt's complete field list is specified in IX.19.

Informative. A Finalizer is deliberately narrow: one enforcement point per permitted operation class — a narrowly scoped database transaction, a single-purpose commit endpoint, a platform-provided ability with server-side permission checks — never a general administrative surface. The breadth of the enforcement point is the breadth of the attack surface; a broad token behind a narrow contract would undo the contract.

IX.8.5 Consent Modes and Execution Capabilities (Normative)

Two consent modes govern automation execution:

1. **Per-action consent** — the existing discipline of IX.5.2, unchanged: one consequential action, one preview, one consent, one PoAE™ record.
2. **Decision-Space Consent** — the operator consents to a declared Decision Space as a standing scope. The client-generated preview (IX.5.3) presents the space itself: the option set, the parameter bounds, the informing data sources, the governing selection rule, and the bound Finalizers — never a vaguer summary and never publisher free text. The consent is Hash-Pinned like every other; any expansion of the space is a consent-class change (IX.5.2).

Capability minting. A consent event mints an **Execution Capability** (IX.2): unforgeable, attenuated to exactly the consented action or space — option set, bounds, Finalizer identity and version, nonce, expiry, idempotency binding — and sender-constrained to the client core's hardware-backed key (IX.10). Finalizers accept proposals only under a valid Execution Capability. No consent, no capability; no capability, no representable Finalizer call. Capabilities are minted, held, and used exclusively by the client core: no rendering or reasoning context SHALL hold, derive, or transit capability material (IX.10(2) extended to Execution Capabilities). A selecting or proposing model consequently holds selection latitude, never execution authority — mechanically, not merely by policy.

IX.8.6 Context Admission — the Closed Model Context (Normative)

Wherever a model participates under IX.8.3 — path tolerance, selection, or proposal composition — its context is itself a contract object:

1. The model context SHALL consist exclusively of three enumerated source classes: (a) manifest-bound prompts and instructions, hash-pinned and publisher-reviewed; (b) consent-admitted data sources, each declared with endpoint, type, and bounds; and (c) operator inputs of the live session.

2. Admission of a new data source is a consent event appended to the existing handshake (change class consent, Hash-Pinned) — never a silent extension.
3. A non-admitted source is not representable as model context; an attempt to introduce one is a deviation (IX.6.5).
4. Where every admitted source is typed and instruction-incapable (numeric, enumerated, and bounded formats), prompt injection is not mitigated but **unaddressable by construction**: no channel exists through which third-party-authored instructions can enter the context. Where an admitted source delivers free text, that fact is itself declared and consent-covered; the residual channel is influence within the agreed space — expansion of the space remains impossible (Annex II §II.2.5) — and that residue is borne by the narrowness of the Decision Space, by Plausibility Verification (IX.8.7), and by the preview at the consequence point. Slot and source types SHOULD prefer instruction-incapable grammars wherever the domain permits.

IX.8.7 Plausibility Verification (Normative)

Between validation and consent, and independently again at the Finalizer, an implementation MAY interpose Plausibility Verification: the check whether a formally valid proposal is sensible given state, history, and context.

1. **Stages.** (a) Rule-based: declarative checks from the manifest or local policy — rate ceilings, value corridors, temporal constraints, state and cross-field consistency — deterministic, versioned, and hash-pinned like the policy engine itself. (b) History-based: comparison against the confirmed execution patterns of the local evidence log. (c) Model-based: advisory review only, permitted exclusively under the ratchet rule below.
2. **The ratchet rule.** Plausibility Verification SHALL only ever increase friction. It may escalate the required PoAE™ tier, place a specific finding into the preview, or block with surfacing. It SHALL NOT lower a tier, substitute for or pre-satisfy any consent, or approve an execution. A wrong, deceived, or compromised plausibility stage can therefore at most delay a legitimate action — never enable an illegitimate one. The enforcement path remains deterministic, and the security statement remains independent of the checker’s intelligence.
3. **Tier modulation, never silent repair.** Findings modulate the consent interaction upward: unremarkable proposals proceed at their declared tier; anomalous proposals step up, with the concrete finding stated in the preview at the consequence point; grossly implausible proposals block and surface. A proposal is never adjusted to pass (no-silent-error), and never executed below threshold merely for being the best available.
4. **Both sides, mutually untrusting.** Client-core verification runs before any capability is used; Finalizer-side verification runs independently (state consistency, rates, publisher-side sanity). Neither relies on the other having run.
5. **Evidence and feedback.** The verdict — rule versions, criteria evaluated, outcome, and any operator override — is recorded in the evidence chain (IX.11). Findings the operator repeatedly overrides are dampened (the meta-feedback discipline of Annex III §5.6 applied to the checker).

Informative. Plausibility narrows the semantic gap — the worst permitted option — it does not close it. Against an adversary operating patiently inside the historical corridor, the standing defenses remain the narrowness of the declared space and the preview at the consequence point.

IX.8.8 Effect Binding (Normative)

The agreement binds the effect, not the code. Compromise of any code, component, model, or client SHALL grant at most the ability to fail to produce the agreed effect, or to trigger it at a legitimate moment — never the ability to produce a non-agreed effect. This follows from the composition of this section: externally observable effects exist only through declared, verified Finalizers (IX.8.4(3)); Finalizers accept only schema-valid, state-bound, capability-carried, consent-evidenced proposals (IX.8.4(2)); and Execution Capabilities are attenuated to the consented space and never held by reasoning or rendering contexts (IX.8.5, IX.10(2)).

Hash verification (IX.4.4) and Effect Binding are two layers against two attacker models, and neither substitutes for the other. Hash pinning defends the honest path — supply-chain integrity, anti-rollback, auditability, exact reproducibility — and supplies the evidentiary chain. Effect Binding defends the compromised path, where local verification can no longer be assumed: the security claim of this Annex does not depend on proving what happened locally, because the consequence is bound where it becomes externally real. Per the claims discipline (IX.0.1): Effect Binding holds by construction of the client–Finalizer contract; the correct, atomic, idempotent server-side implementation of declared Finalizers is a trust assumption (IX.12.3) and its validation an objective pending (IX.12.2).

IX.9 Operating Policies: Ground State and Secure Browse

The client's ground state is fail-closed: only verified, declared, cryptographically bound WR content and functions exist within it. This version of the client is WR-only — the strictest policy is the default, not an opt-in hardening; the open web is not reachable through the client at all (IX.13, Profile A).

Secure Browse names the policy tier for managed deployments: an organization MAY additionally constrain which publishers, attestation levels, origins, functions, and data classes its operators may use, importing policy through the local policy engine. Under Secure Browse, the structural-growth tier split of IX.5.2 applies (silent → notice), and Assisted Code Capture (IX.3.1) MAY be disabled or downgraded to notice-then-manual-confirmation by policy; minimum Channel Provenance requirements MAY additionally be imposed on assisted_email offers (e.g. suppressing offers from channels failing DMARC, forcing manual entry) and on external-link opening from unauthenticated channels (up to blocking it, per IX.3.1 rule 8) — friction-increasing constraints conforming to the ratchet discipline of IX.3.1 rules 7–8; scan and manual entry are never removable, and the unverifiable-sender warning is never removable by policy either.

Where a future implementation profile adds general web browsing (IX.13, Profile B), the following invariant is normative in advance: **the WR security model is never weakened by the presence of an open-web context.** The difference between a permissive and a restrictive policy is exclusively whether non-WR content is reachable outside the WR trust domain; inside it, identical rules apply always. Secure Browse under Profile B means: non-WR web content is not reachable at all.

IX.10 Isolation and Key Custody (Normative, Engine-Agnostic)

Regardless of implementation profile:

1. Handshake keys, consent capture, contract state, and the policy engine SHALL reside in the client core, outside every rendering context, with platform hardware-backed key storage used where available.
2. No rendering context SHALL hold, derive, or transit key material or consent authority. Compromise of a rendering context SHALL yield no WR authority — no key, no consent, no contract mutation, no store write. Store writes occur only through the client core after hash verification (IX.6.2).
3. Where any profile hosts both WR and non-WR contexts, they SHALL be separated at the strongest available platform boundary (separate processes and data domains at minimum), with no shared storage, cookies, caches, or session state. Compromise of a non-WR context SHALL yield no WR authority.
4. All consent events, deviations, handshake formations, manifest version transitions, and automation runs SHALL be recorded in the client's append-only local evidence log (IX.11).
5. **Modular separation of stages.** The client and its upstream pipeline SHALL separate preparation, normalization, manifest generation, manifest verification, policy enforcement, and rendering as distinct components with defined interfaces. The rendering module consumes, through its interface, exclusively material that has already passed manifest verification and policy enforcement; it contributes no verification decision, and no verification decision SHALL depend on its internals. Rendering is intentionally designed as a replaceable module: a conforming deployment MAY substitute the rendering module without altering any normative property of IX.4–IX.9. Correspondingly, the renderer is not the primary security boundary of this architecture and SHALL NOT be represented as such; security is established by the surrounding preparation and verification pipeline. Hardening of the rendering module is defense in depth, never the load-bearing mechanism — a renderer presented only with verified, policy-conforming, execution-inert input has correspondingly little latitude to be exploited into violating the approved state.

IX.11 Auditability and Evidence (Normative)

For every rendered WR View and every executed automation, the client SHALL retain: the Handshake Contract reference and version; the Site Manifest version and its root chain; the hash-bound resource set actually rendered; slot values bound (or their commitment, per data-class policy); WR Link activations with their bound definitions; consent records (PoAE™) and previews as presented (hash-pinned); deviations with cause; and recording runs with their pins and outcomes. Given these, what was shown and what was executed, under which contract and rules, SHALL be demonstrable after the fact — the presentation- and execution-layer instance of the corpus's proof discipline. This is also the specified answer to transient-compromise concerns (informative): the contract model cannot prevent every attack, but it makes the executed state provable and the declared state the only executable one.

IX.12 Security Model, Assumptions, and Residual Risk

IX.12.1 Properties by Construction (Normative claims)

- No undeclared resource can be fetched at render time: the rendering context has no network path (IX.6.2).
- No active content executes in a WR View: script-class capability, timers, and asynchronous execution interfaces are excluded from the subset and disabled at the engine boundary (IX.6.1, IX.6.7).
- No content renders without a prior hash match against a signed, monotonic, freshness-bound manifest (IX.4.2, IX.6.3); every such hash is a SHA-256 digest verified unconditionally before processing, and a validly signed manifest with mismatching content does not render (IX.4.4).
- No WCR recording executes if its digest, its pin, or any resource it depends on fails verification against the approved manifest; modified recordings are rejected (IX.4.4, IX.8.1).
- A rendered WR View cannot evolve beyond the previously approved manifest state: no capability through which post-verification change could be effected exists within the WR Trusted Execution Domain (IX.6.7).
- No automation executes outside its declaration, its pin, and its consent (IX.8).
- No formation path exists besides the capture methods of IX.3.1 — WR Code® scan, manual entry, and Assisted Code Capture — all converging on the identical verification-and-consent path; no distributed link, code, or invitation carries authority; a captured code — scanned, typed, or client-recognized — remains untrusted input until the complete client-side verification chain has succeeded (IX.1.5, IX.3.4, IX.3.5); capture, including assisted capture, confers no authority, and formation occurs only on Hash-Pinned Consent. No hyperlink can form, extend, or upgrade a handshake.
- Channel provenance informs and never authorizes: a channel verdict (SPF/DKIM/DMARC, sender/publisher alignment) evaluated at depackaging can only ever increase friction at capture — suppress or downgrade a Connect offer, or place a stated finding in the preview — never upgrade trust, reduce friction, or substitute for any step of the verification chain (IX.3.1 rule 7; the ratchet discipline of IX.8.7 applied to capture). Verdicts are surfaced uniformly at every channel-consequential decision point, including the external-link risk dialog, and an unauthenticated channel — DKIM and DMARC absent or unverifiable — is always visibly flagged as sender-unverifiable, with the warning unsuppressible by any non-policy mechanism (IX.3.1 rule 8). The local fraud analysis consumes the record only additively: its findings can add friction on top of the deterministic verdicts, never suppress, soften, or replace them, and never upgrade trust (IX.3.1 rule 9).
- Consent records permanently describe what was consented to, because consent attaches to the envelope and to definitions, not to mutable content (IX.5.1).
- No externally observable consequence exists outside a declared, hash-verified Finalizer, and no Finalizer call is representable without a valid, attenuated Execution Capability minted by consent (IX.8.4, IX.8.5).

- The security statement is independent of the intelligence of the path: selection latitude never carries execution authority, and a compromised or deceived upstream component — including any plausibility stage — can at most withhold or delay the agreed effect, never produce a non-agreed one (IX.8.3, IX.8.7, IX.8.8; under the trust assumptions of IX.12.3).
- Where every admitted model-context source is typed and instruction-incapable, prompt injection is unaddressable by construction (IX.8.6).
- Every consequential effect — of a web-surface automation and, from v2.0, of a computer-use action — passes the five-phase lifecycle Validate → Execute → Witness → Seal → Anchor (IX.17): validity is complete at Seal; anchoring is asynchronous and evidentiary only; and the executed effect is bound by Intent Hash to the presentation the operator actually confirmed, so any divergence between displayed and executed action is an invalid consent record and a deviation (IX.19.2).
- The consent assurance level is itself evidence: the Consent Mechanism Class (C0–C4) and its cryptographic artifact are bound into the sealed record, the used mechanism must meet or exceed the policy-declared minimum, and no publisher or checker can lower it (IX.18); sealed receipts form a per-contract append-only hash chain in which removal, reordering, or insertion is cryptographically detectable at Tier L alone (IX.19–IX.20).
- Wherever a model participates in execution, it routes over pre-declared, cryptographically pinned paths: it selects, parameterizes, and branches within declared constraints, and no routing decision can create, extend, or modify a path (IX.16.4, IX.21.5).

IX.12.2 Objectives Pending Validation (Informative)

Resistance of the residual parser surface (image decoding of normalized derivatives, styling and layout parsing, font handling) to malformed input; correctness and reproducibility of the normalization pipeline; robustness of the interception layer across engine versions; behavior under hostile platform conditions; correct, atomic, and idempotent server-side implementation of declared Finalizers, including state binding and replay resistance (IX.8.4); the operational safety of advisory model-based plausibility stages under the ratchet rule (IX.8.7); the detection quality and false-positive discipline of Fraud Watchdog™ under its advisory, ratchet-conforming role (IX.3.1 rule 9); the correctness and durability of the Tier C transparency log and the Tier X anchoring pipeline under their evidentiary-only role (IX.20); and the completeness and reliability of pre-/post-state witnessing across the Transition Classes of IX.21.3, in particular for observed_effect targets. These require implementation, testing, adversarial analysis, and audit before being claimed. The layered posture is: normalization upstream removes malformed structure before content reaches the device; hash-before-decode ensures only pipeline-produced artifacts are decoded; and a decode-stage exploit that fires regardless lands in a rendering context that holds no keys and no authority (IX.10). This is reduced attack surface by construction and contained residue by platform — not invulnerability.

IX.12.3 Trust Assumptions (Informative)

The publisher root and attestation chain (Annex VII/IX); the Optirando™ normalization instance as a supply-chain-critical component (isolated, versioned, tool-governed, auditable — IX.6.3); platform integrity of the client device (hardware keystore, process isolation); WebPKI/TLS for the origin verification channel; the correct, atomic, idempotent server-side implementation of declared

Finalizers (IX.8.4, IX.8.8); and, for the verification tiers of IX.20, the key custody of the Tier C transparency log operator and the liveness and immutability properties of the Tier X anchoring ledger — noting that per IX.20 neither tier is validity-conferring, so their compromise degrades independent verifiability, never execution safety. A compromised publisher pipeline producing signed-but-malicious normalized content is detected by none of the above and is mitigated only by the passivity of the subset, the slot discipline, and auditability.

IX.12.4 Non-Goals (Normative)

The system bounds technical execution, rendering, and permitted actions. It does not and cannot guarantee that publisher statements are true, that offered goods are of merchantable quality, that real-world obligations are fulfilled, or that the world matches the representation. The architecture reduces technical attack and manipulation surface; it replaces no assessment of truth, quality, or real-world performance.

IX.13 Implementation Profiles (Informative)

The normative core of this Annex (IX.3–IX.12, IX.16–IX.21) is engine-agnostic by design: enforcement is specified as behavior — out-of-band fetch, verified store, render-time starvation, hash-before-decode, interception, hard stop — with no dependency on a particular engine, and the modular-separation invariant of IX.10(5) fixes the renderer as a replaceable module behind a stable interface. Engine choice is an implementation profile; migrating between profiles changes no normative statement, precisely because no security property rests on renderer internals.

Profile A — system-engine-hosted WR-only client (first implementation, mobile). The client is a standalone app embedding the platform's system web engine (auto-updated by the platform vendor independently of the app) as a passive renderer: script-class capability disabled at the engine API, all resource resolution intercepted and served from the Verified Local Store, client core and evidence log in the app process, keys in hardware-backed storage. The client is WR-only: it is a verified-execution client, not a general-purpose browser, and is not represented as one. Platform note: on iOS the system engine is mandated outside specific jurisdictions; Profile A is the durable iOS profile, which is one reason the enforcement layer, not the engine, carries the security claim.

Profile B — full-engine browser product (later stage, staffed). A maintained full-engine browser (e.g. a Chromium-derived build) adding a general-browsing Normal Mode alongside the WR trust domain, under the advance invariants of IX.9 and IX.10(3). This profile carries an explicit operational obligation: tracking upstream engine security releases with a published patch cadence. Product naming for Profile B is a business decision outside this specification.

IX.14 Staged Roadmap (Informative, stages marked)

- **Stage 1 (this version):** Profile A; raster images only (WebP-normalized); no video; no general web; no general capability/tool exposure beyond declared automations; Secure Browse policy tier for managed deployments.
- **Stage 2 (planned):** additional normalized media classes; richer slot types; broadened WR Link surfaces.

- **Stage 3 (planned):** video via the normalize-hash-verify pattern with progressive reconstruction (Annex V discipline).
- **Stage 4 (planned):** Profile B with Normal Mode under the IX.9/IX.10 invariants.
- **Research direction (not planned for a numbered stage):** general capability declaration for non-web providers (APIs, local services) under the same declare-bind-limit-consent-block pattern; taken up only when its security analysis can be carried in full.

IX.14.1 Architectural Outlook — Hash-Bound Website & Automation Builder (WR Builder) (Informative)

This subsection describes a potential future extension of the WR architecture. It is presented explicitly as an architectural outlook, not as an implemented feature and not as a required component; nothing in IX.3–IX.12 depends on it, and it introduces no normative obligation.

The trust model of this Annex fixes the rendering and interaction state: operator and publisher agree, through the signed Site Manifest, on exactly what is displayed and exactly which declared automations may run against it. The same manifest-driven principles admit a natural extension one level deeper. The **WR Builder** is conceived as a web builder with a prefabricated component library: forms — registration and input flows in particular — interaction patterns, and automations exist as pre-built, WR-conforming components, each canonical in form, openly specified, and hash-bound. A publisher does not author free executable code against the WR surface; it composes its site and its automations from these components and parameterizes them. The builder then emits, in one build pass, two coupled outputs with a deliberately sharp boundary between them:

1. **The composed automation container.** The composition — every component instance, its configuration, and the exact executable code realizing it — is transformed at build time into a canonical representation and emitted as an openly specified container: the container format itself an open specification, and, for public offerings, its contents publicly inspectable and auditable in the sense of Annex IV §IV.6.3(1) — what a container authorizes to execute is exactly as open to inspection as what composes an operator's WR Views. The container is produced alongside the WCR recording in the same build pass, so that the semantic recording and the exact code authorized to realize it are generated and bound together. The complete composition, including the exact executable code and the precise execution sequence of the automation, is then summarized in the signed Site Manifest: the container, each component instance, and the step sequence identified and bound by SHA-256 digest (IX.4.4). Before execution, the WR Execution Client verifies the container and its components against the approved hashes; only matching code executes.
2. **Everything outside the container is normalized.** Content beside the container carries no exactness guarantee and requires none: it traverses the ordinary normalization discipline of IX.6.3 — media elements, in particular, are replaced by normalized representations that are visually and informationally equivalent to their sources but are new files carrying none of the original byte structure. The boundary is the point of the design: inside the container, byte-exact canonical code under hash verification, because execution demands exactness; outside it, reconstructed derivatives under the same manifest, because rendering demands only equivalence. Exactness is spent precisely where it is load-bearing.

This is the composition discipline of Annex VIII (VIII.3.3) applied to the execution layer. What Annex VIII establishes for visual artifacts — composition from attested building blocks, never

synthesis at the moment of consequence — the WR Builder would establish for executable web functionality: the security floor of the execution layer is governed by how tightly the executable space is constrained (the role the layout grammars of VIII.5.3 play for composition quality), not by how carefully free-form code is reviewed after the fact. Prefabricated forms and registration flows are the canonical case: exactly the interactive functions that strict passivity removes from the WR View (IX.6.1) return as declared, prefabricated, hash-verified components under consent — never as arbitrary site code.

Seen through the consequence-boundary model of IX.8.3–IX.8.8, the WR Builder's actual product is the **action contract**: declared actions and Decision Spaces, Finalizer bindings, preview and receipt schemas — discover, constrain, preview, finalize, receipt. The component library is the authoring surface through which publishers produce that contract; the contract is the product, the interface is access. This is also what makes the builder platform-independent: action contracts are generatable from any sufficiently described execution surface, not only from prefabricated components.

The open extension path. The component library is the primary authoring path, not a boundary. Because the builder, its component library, and its container format are open, a publisher MAY integrate any other automation of its choosing — including free, hand-written code — through the same build pass. The WR Desk™ runtime is deliberately indifferent to authoring provenance: it is interested exclusively in the manifest and its enforcement. Completion under either path yields the identical artifact discipline: the signed Site Manifest carries the exact executable code and the precise step sequence of the automation, SHA-256-bound (IX.4.4) and verified before execution — only matching code runs, whoever wrote it.

The closed action set at the execution boundary. The openness of the authoring path must never widen the execution surface. Regardless of what is authored — library composition or free code — the execution path would effect only defined, permitted actions: a closed, declared action vocabulary enforced at the renderer/execution boundary, in exactly the sense of IX.8.2 (only declared steps execute) and under the register discipline of Annex VIII §VIII.6.5.3 — "arbitrary action" is not, and never becomes, a vocabulary entry. Free code therefore differs from a library component in authorship and in audit effort, never in capability: whatever its interior logic, its externally effective operations are expressible only within the permitted action set; an operation outside the vocabulary is unrepresentable as an execution request, and any attempt is a deviation with hard stop (IX.6.5). The division of labor in the enforcement stack is exact: hash verification establishes which code runs (IX.4.4), the closed action vocabulary bounds what any code can effect, and the PoAE™ consent decides whether it runs at all. This is also what preserves the preview contract for free code: because effects are drawn from a declared, closed vocabulary, the client-generated preview (IX.5.3) can enumerate them precisely regardless of who authored the code.

Conceptually, this extends the existing recording concept from agreement on the expected rendering state toward agreement on the exact executable code authorized to produce that state. Every property of the present architecture would carry over by construction: modifications to the container or to any component would become cryptographically detectable exactly as modifications to recorded resources are today (IX.4.4); execution would be manifest-governed and default-deny exactly as rendering is (IX.6.2); deviation would hard-stop (IX.6.5); and the verifying pipeline, not the executing component, would remain the locus of security (IX.10(5)). Because the guarantee is established by verification rather than by containment, such an extension is of particular interest on

platforms where strong process- or hypervisor-level isolation is unavailable (IX.1): determinism by construction bounds what can execute where isolation cannot bound what execution can do. Where the builder's transformation — like the ordinary transformation of IX.6 — leaves a site non-functional in WR form, the complementary mechanism is the WR Compatibility Path (IX.6.8): the publisher authors a dedicated representation, and the strictness of the model is never weakened to accommodate the site.

Two clarifications keep the outlook within the claims discipline of IX.0.1. First, hash verification of an executable artifact would be a necessary condition for execution, never a sufficient one: the PoAE™ consent semantics governing automation execution remain in force unchanged, and verification neither replaces nor pre-satisfies any tier (Annex II §II.2.5; VII-10.1). Second, no implementation architecture is implied; the purpose of this subsection is solely to record that the WR trust model — canonical form, SHA-256 binding, signed manifest, unconditional pre-processing verification, default-deny, hard stop — is not specific to rendering, and could later be extended from deterministic rendering toward deterministic code execution while preserving the same manifest-driven security principles. The name WR Builder is registered for this concept to prevent divergent usage; the registration carries no implementation claim.

IX.14.2 Relation to Prior Work and Claimed Contribution (Informative)

The constituent mechanisms of this Annex are established practice and are not claimed as novel. For the completeness expected of the research corpus, the closest neighbours in the web space are named here, together with what each does and does not cover:

- **Subresource Integrity (SRI)** establishes per-resource hash pinning in web content: fetched resources are bound to declared digests. SRI operates inside a live page whose network context remains open, protects only resources the author chose to pin, and carries no consent, versioning, or governance semantics.
- **Content Security Policy (CSP)** establishes declarative capability restriction — constraining script execution and network destinations — enforced by the client against a policy the site itself declares. The policy author is the party being constrained; nothing binds the policy to an operator-side consent record.
- **Signed HTTP Exchanges and Web Bundles (Web Packaging)** establish origin-signed, self-contained, offline-verifiable web content: a page renderable without contacting its origin, with its authorship cryptographically attributable. They provide signed content, not a governed relationship: no scope envelope, no change classes, no consent binding, no execution model.
- **AMP-class publisher parallel formats** (AMP; proprietary analogues such as Facebook Instant Articles and Apple News Format) establish the publisher-authored second rendering path: an alternative representation maintained in the publisher's codebase, authored against a validated, restricted subset, delivered through a controlled runtime. This is the closest neighbour of the WR Compatibility Path (IX.6.8); it is neither hash-bound to a signed monotonic manifest, nor consent-governed, nor rendered under render-time network starvation.
- **Component-based site builders** (low-code / no-code web platforms) establish composition of sites from prefabricated components — forms, flows, interaction patterns — instead of

free authoring. The code such components actually execute is, however, neither canonicalized, nor declared to the visitor, nor verifiable by the visitor's client; the composition constrains authoring convenience, not the executable space. This is the closest neighbour of the WR Builder's component model (IX.14.1).

- **Browser reader modes** establish client-side reduction of pages to passive reading representations — but heuristically, decided by the client, neither publisher-declared nor hash-bound, with no equivalence contract on either side.
- **Reproducible builds and binary transparency** establish canonical build artifacts whose digests verify exactly what runs, and platform code signing establishes that only signed artifacts execute. These are the closest neighbours of the WR Builder outlook (IX.14.1); they verify artifact identity and publisher identity, not a per-operator, consent-bound authorization of the specific code within a declared scope.
- **Web archiving formats (MHTML, WARC)** establish frozen page snapshots — unsigned, ungoverned, without freshness, consent, or replay semantics.

Each neighbour supplies a mechanism; none supplies the contract. The claimed contribution of this Annex is the normative composition of these mechanism classes under the corpus's governance model: entry restricted to code-based initiation completed only by Hash-Pinned Consent; a two-object contract that keeps consent records permanently accurate under content change; rendering exclusively from a verified local store under render-time network starvation — where SRI and CSP restrict behavior within a live network context, the WR View removes the network from the rendering context entirely; a publisher-declared, never heuristic, reduced representation whose every element is hash-bound to a signed, monotonic, freshness-bound manifest; change-class governance that scales operator interaction to consent relevance; a deterministic execution domain serving as the containment substitute on platforms without kernel-level isolation under the deployment's control; the Compatibility Path as the governed accommodation mechanism that keeps the model's strictness invariant; and automation confined to declared, version-pinned, individually consented recordings with PoAE™ proof — with no path from any of it to execution authority. Individually, these ideas have neighbours; to the author's knowledge, following review of the adjacent work above, no existing system composes them in this governed, consent-bound form. The corresponding claimed contribution of the execution model added in v2.0 is stated in IX.23.

IX.14.3 Publisher Adoption Path — Low-Friction Integration (Informative)

The strictness of this Annex lives on the client side and in the normalization pipeline; nothing in it requires the publisher to rebuild its web presence. Adoption is therefore designed as a ladder of strictly additive levels, in which each level is complete and useful in itself, publisher effort varies by level, and client-side enforcement never varies at all:

- **Level 0 — Generated manifest, read-only (hours, no code change).** The publisher points the WR onboarding tooling (self-run, or operated as a service on Optirando™ infrastructure) at its existing site. The tooling crawls the declared origin set, normalizes content per IX.6.3, maps what fits into the Pinned Rendering Subset, flags what does not (which degrades per the ordinary rules rather than blocking adoption), and emits a draft Site Manifest. The publisher reviews, signs under its Manifest Root, publishes the Discovery Record and the well-known origin endpoint (IX.3.5) — and has a conforming, read-only WR View. Nothing in the site's codebase changes.

- **Level 1 — Annotated dynamics (small, localized changes).** Declared slots (IX.6.4) and WR Links (IX.7) are introduced through lightweight annotations in the existing markup at template level; the onboarding tooling derives the corresponding manifest declarations from the annotations on the next generation run. Live values and consent-gated functions become available without restructuring the site.
- **Level 2 — Composed automations (no custom code).** Automations — registration forms, input flows, service actions — are added from the WR Builder component library (IX.14.1): prefabricated, parameterized, manifest-summarized with exact code and precise sequence, consent-admitted per IX.5.2. Publishers gain the automation surface without writing executable code.
- **Level 3 — Dedicated representation (full authoring).** Where the ordinary representation does not survive transformation, or where the publisher wants the full surface deliberately, the WR Compatibility Path (IX.6.8) or a fully WR-Builder-authored representation provides the designed-for-the-subset path.

Maintenance follows the same low-friction logic: manifest generation is repeatable and belongs in the publisher's ordinary build or deployment step — the site changes, the tooling regenerates, the version increments, and the change classes of IX.5 route exactly the operator interaction the change deserves (most routine changes being silent by design). Two properties keep the ladder honest: levels are additive, never alternative trust regimes — a Level-0 publisher and a Level-3 publisher are verified, rendered, and enforced identically; and no level relaxes any rule of this Annex — the ladder varies what the publisher invests, never what the operator's client enforces.

IX.15 Open Specification Items (Informative)

Formal grammar of the Pinned Rendering Subset (markup and styling whitelist, positive construction); reproducibility criteria for the normalization pipeline (deterministic transformation and re-verification); Site Manifest schema and hash-tree layout within the Annex VII artifact machinery; freshness assertion format and offline grace semantics; slot type system and bounds language; Discovery Record format; declaration format, authoring rules, and validation tooling for the WR Compatibility Path (IX.6.8); notice-class UX requirements; evidence-log retention and export format; Decision Space declaration language (option sets, bounds language, selection-rule format); Execution Capability token format and attenuation fields; Finalizer effect schemas and the execution-receipt format; the state-binding profile for Action Proposals (candidate basis: standardized measured-component attestation models); the plausibility rule language and history-corridor parameters; the manifest signature envelope serialization; Profile B process/site-isolation requirements. Added in v2.0: Execution Receipt serialization and signature envelope; transparency-log countersignature and inclusion-proof format (Tier C); Merkle batching profile and anchor-transaction format (Tier X); Transition-Class witnessing profiles per target class (filesystem, process, configuration, device); Consent Mechanism Class artifact formats (WebAuthn assertion carriage, attestation evidence carriage); the C0 policy-rule admission schema; and the computer-use chain declaration format and catalog admission rules (IX.21.5).

IX.16 The WR Trusted Execution Domain — Generalized Execution Authority

IX.16.1 Scope Extension (Normative)

This section block (IX.16–IX.23) extends the scope of this Annex: it specifies the execution semantics of the WR Trusted Execution Domain — the phase model, consent mechanism classes, proof record, and verification tiers under which every consequential effect of this Annex is executed and evidenced — and generalizes the consequence boundary of IX.8 from contract-bound web interaction to computer-use execution (file operations, process invocation, application control, and system configuration). Nothing in this block modifies, relaxes, or re-defines any rule of IX.0–IX.15; where this block restates an existing rule, the restatement is a phase-model view of the identical rule, and the existing section remains authoritative. Wire formats, capsule serialization, cryptographic primitives, and transport remain defined exclusively by the Canon and its normative Annexes, per IX.0. The claims discipline of IX.0.1 applies to this block without exception.

IX.16.2 Definitions (Normative)

Terms defined in IX.2 apply unchanged. This block adds:

WR Trusted Execution Domain (WR TED), generalized. The definition of IX.2 is extended, not replaced: beyond the web surface, the WR TED designates the client-enforced boundary within which any consequential execution of the Optirando™ system occurs under the consequence-boundary model of IX.8. **The WR TED is not an operating system. It is a cryptographically governed execution authority that may be implemented using existing operating-system security mechanisms or future dedicated runtime environments.**

Intent Hash. The SHA-256 digest (IX.4.4) of the action presentation exactly as rendered to the operator at the consent event — the client-generated preview per the preview contract of IX.5.3, in its presented form. The Intent Hash is bound into the Execution Capability at minting and into the PoAE™ record at sealing, establishing the What-You-See-Is-What-You-Execute binding (IX.19.2).

Transition Class. The typed classification of a consequential state transition by the atomicity and observability properties of its target: atomic, journaled, or observed_effect (IX.21.3). The Transition Class is declared per Finalizer effect schema and recorded in the Execution Receipt.

Consent Mechanism Class. The normative enumeration C0–C4 (IX.18) of the mechanism through which a consent event was performed, recorded verifiably in the PoAE™ record together with the mechanism's cryptographic artifact where one exists.

Verification Tier. One of the three evidence-verification levels L, C, X (IX.20) at which a sealed Execution Receipt is made verifiable: local hash chain (L), countersigned central transparency log (C), external public-ledger anchoring (X).

Transaction Envelope. The execution wrapper for computer-use effects (IX.21): declared target objects, pre-state witnessing, the declared transition, post-state witnessing, and the sealed receipt, executed under a declared Transition Class.

IX.16.3 Position — Execution Authority, Not Platform (Normative)

The WR TED holds exactly the following responsibilities: verification of Execution Capabilities and consent evidence; deterministic execution of declared transitions through declared Finalizers; witnessing of pre- and post-states; sealing of PoAE™ records and Execution Receipts; and submission of evidence to the configured Verification Tiers. Everything else — rendering, reasoning, planning, selection, proposal composition, user interaction — lies outside the domain and holds no execution authority (IX.8.5, IX.10). The domain does not schedule processes, manage memory, or arbitrate hardware; it governs authority over consequences, and it MAY be realized on top of platform security mechanisms (IX.22) without itself becoming one.

IX.16.4 The AI as a Router (Normative)

Wherever a model participates in execution under this Annex (IX.8.3), its role is that of a **router over pre-declared deterministic paths**. The paths — WCR recordings, Decision Spaces, declared automations, declared computer-use chains (IX.21.5) — are defined in advance, per handshake between the parties or per catalog admission in the deployment, and are cryptographically pinned before the model ever runs. Routing comprises exactly: interpreting the situation, selecting the applicable declared chain, binding typed parameters within their declared bounds, and choosing among declared branches — each within the declared constraints. No routing decision SHALL create, extend, or modify a path; every route halts at every PoAE™ gate; and selection latitude never carries execution authority (IX.8.5). Intelligence lies in choosing well among contracted paths; authority lies entirely in the paths themselves. **The AI routes; it never paves.**

IX.17 The Finalizer Execution Lifecycle (Normative)

Every consequential, externally observable state transition — of a web-surface automation per IX.8.4, and of a computer-use action per IX.21 — SHALL pass through the following five phases in order. Phases 1 and 2 are the phase-model restatement of IX.8.4(1)–(3), which remain authoritative; phases 3–5 make explicit what IX.8.4(4) and IX.11 establish and extend them normatively.

1. **Validate.** The Finalizer verifies, fail-closed: the Execution Capability (validity, attenuation to the consented action or Decision Space, sender constraint to the client core, nonce, expiry); the consent evidence including the Consent Mechanism Class against the policy-required minimum (IX.18.3); the Intent Hash binding; the Action Proposal against the declared effect schema and bounds; the target Finalizer version against the verified manifest state (anti-rollback, IX.8.4(1)); idempotency by proposal hash; and freshness. Any failure aborts before any effect, with no partial execution and no silent continuation.
2. **Execute.** The authoritative state transition is performed exclusively by the Finalizer, deterministically within the consented scope, under the expected-state binding of IX.8.4(2): the pre-state binding is compared against current state and the transition committed atomically only on match (compare-and-swap for atomic targets; the class-specific disciplines of IX.21.3 for journaled and observed_effect targets). On mismatch: abort, never adapt.
3. **Witness.** The pre-state and the post-state of every declared target object are captured as SHA-256 digests over their canonical or declared observable form. For a database row under an atomic Finalizer, witnessing is the expected-state binding and resulting-state binding

already required by IX.8.4; for computer-use targets, witnessing follows the Transaction Envelope discipline of IX.21. A transition whose witnessed post-state diverges from the declared effect is a deviation (IX.6.5).

4. **Seal.** The Finalizer signs the PoAE™ record and returns the signed Execution Receipt (field list in IX.19.1), which the client core verifies and appends to the local evidence chain. Sealing completes validity: a sealed receipt is valid evidence at Tier L without any further step.
5. **Anchor.** The sealed receipt is made verifiable at the configured Verification Tiers (IX.20). Anchoring is asynchronous and evidentiary only: it SHALL NOT be a precondition of execution, SHALL NOT confer or complete validity, and its absence, delay, or failure SHALL NOT invalidate a sealed receipt — it removes only the corresponding independent verifiability.

IX.18 Consent Mechanism Classes (Normative)

IX.18.1 Enumeration

Consent events under this Annex are performed through exactly one of the following mechanism classes:

- **C0 — policy-autonomous.** No interactive consent at execution time: a previously admitted policy rule authorizes the action class, and the policy decision record replaces the interactive event. Admission of the policy rule itself is an ordinary consent event (IX.5.2, class consent) with its own Hash-Pinned preview; C0 never arises silently, and nothing in C0 pre-satisfies any tier for actions outside the admitted rule (Annex II §II.2.5).
- **C1 — UI confirmation.** The explicit operator tap or click in client chrome — the consent tap of VII-10.1.
- **C2 — Passkey / WebAuthn.** Consent performed through a WebAuthn assertion (platform or roaming authenticator, user verification required); the assertion itself is carried as cryptographic evidence in the PoAE™ record.
- **C3 — hardware security key 2FA.** Consent performed through a dedicated hardware security key as a second factor; the key’s response artifact is carried as evidence.
- **C4 — hardware attestation.** Consent bound to a hardware attestation of the executing environment (platform attestation, measured state); the attestation evidence is carried in the record.

IX.18.2 Artifact Binding

Where a class produces a cryptographic artifact (C2–C4), that artifact — or its SHA-256 digest with the artifact retained in the evidence log — SHALL be bound into the PoAE™ record, so that not merely the occurrence of consent but its assurance level is verifiable after the fact.

IX.18.3 Meet-or-Exceed (Normative)

Local policy declares, per PoAE™ tier and per action class, the minimum required Consent Mechanism Class. The mechanism actually used SHALL meet or exceed that minimum; a lower class is a validation failure (IX.17 phase 1). Plausibility Verification MAY escalate the required

class for a specific proposal under the ratchet rule (IX.8.7) — it SHALL NOT lower it. A publisher SHALL NOT be able to require a lower class than the operator's policy demands; a publisher MAY declare a higher minimum for its own automations.

IX.19 The PoAE™ Record and the Execution Receipt (Normative)

IX.19.1 Field List

The Execution Receipt of IX.8.4(4) is specified in full. A sealed PoAE™ record / Execution Receipt SHALL bind, at minimum:

1. the Execution Capability token identifier;
2. the consented scope reference (action identifier or Decision Space identifier) and Handshake Contract reference;
3. the policy decision reference (including the C0 policy rule where applicable);
4. the Finalizer identity and version;
5. the target object identifier(s);
6. the pre-state hash(es) and post-state hash(es) (witnessing, IX.17 phase 3);
7. the Intent Hash (IX.19.2);
8. the Transition Class (IX.21.3);
9. the Consent Mechanism Class and its artifact or artifact digest (IX.18.2);
10. the proposal hash (idempotency key, IX.8.4(2));
11. the timestamp;
12. a strictly monotonic per-contract sequence number; and
13. the SHA-256 digest of the previous receipt under the same contract (hash chaining).

Fields 12–13 make the receipt stream an append-only hash chain: removal, reordering, or insertion of receipts is cryptographically detectable at Tier L without any external party (claims discipline per IX.4.4(4): detectable, not "tamper-proof").

IX.19.2 The Intent Hash — What You See Is What You Execute (Normative)

At every interactive consent event (C1–C4), the client core SHALL compute the Intent Hash over the client-generated preview exactly as presented (IX.5.3) and bind it into the minted Execution Capability; the Finalizer SHALL bind the identical Intent Hash into the sealed record. It is thereby provable that what was executed is what was displayed and confirmed — the display-versus-execution gap, the core manipulation vector of agentic systems, is closed evidentially: any path on which the executed action differs from the presented action produces a receipt whose Intent Hash does not resolve to the presented preview, which is an invalid consent record under Hash-Pinned Consent (IX.3.4) and a deviation (IX.6.5). For C0, the Intent Hash binds the hashed preview of the admitted policy rule.

IX.20 Verification Tiers L / C / X (Normative)

A sealed Execution Receipt is verifiable at up to three tiers:

- **Tier L — local.** The append-only, hash-chained local evidence log (IX.11, IX.19.1). Tier L is always present and requires no network, no server, and no third party.
- **Tier C — central transparency log.** The client MAY submit receipt digests to a countersigned transparency log operated on Optirando™ infrastructure. The log receives and stores hash commitments only — never previews, slot values, target contents, or any operator data — countersigns them with its own key and inclusion proof, and returns the countersignature into the local evidence chain. Tier C adds an independent, time-ordered existence witness under a second key holder.
- **Tier X — external anchoring.** Receipt digests are aggregated into Merkle trees and the roots anchored on a public distributed ledger. Solana is the named anchoring embodiment; the mechanism is ledger-agnostic and MAY be realized on any ledger providing durable, publicly verifiable inclusion (the anchor-provider abstraction of the Canon's PoAE™ machinery). Only Merkle roots — hash commitments of hash commitments — leave the deployment; anchoring is batched and asynchronous.

Normative invariants across tiers: (1) validity is established at Seal (IX.17 phase 4); Tiers C and X add independent verifiability and never validity — anchoring is never a precondition of execution, and absence, delay, or failure of a higher tier never invalidates a sealed receipt and never blocks or delays the consequence path; (2) tier selection is local policy plus operator choice, and a publisher SHALL NOT be able to force a lower tier than the operator's policy demands nor require a higher tier as a condition of service beyond its declared automation definitions; (3) no tier transmits operator content off-device — the local-evidence discipline of IX.11 is unchanged, and Tiers C and X operate exclusively on digests; (4) higher tiers confer no authority: a receipt anchored at Tier X authorizes nothing that its consent did not (Annex II §II.2.5).

IX.21 The Transaction Envelope for Computer-Use (Normative)

IX.21.1 Generalization

The consequence boundary of IX.8 is not specific to the web surface. Computer-use execution — file operations, shell and process invocation, application control, system and OS configuration, and hardware-interfacing operations — SHALL be governed by the identical model: declared paths, consent-minted attenuated capabilities, declared Finalizers, the five-phase lifecycle (IX.17), sealed receipts (IX.19), and tiered verification (IX.20). The proof chain of a computer-use action is built exactly like the proof chain of a web automation.

IX.21.2 The Envelope

Every consequential computer-use effect SHALL execute within a Transaction Envelope declaring: the target object(s) by typed identifier (file path class, process class, configuration key class, device class); the declared transition (from the closed action vocabulary — IX.8.2 applied beyond the web surface: only declared steps execute, and "arbitrary action" is never a vocabulary entry); the

Transition Class (IX.21.3); the pre-state witness; and the expected post-state. Pre-state witnessing occurs before the transition; post-state witnessing after it; both enter the receipt (IX.19.1(6)). A post-state diverging from the declared expected effect is a deviation with hard stop (IX.6.5).

IX.21.3 Transition Classes

- **atomic.** The target supports transactional commit (database, transactional store, atomic file replace). Discipline: compare-and-swap on the witnessed pre-state; commit or abort, never partial.
- **journaled.** The target supports staging with rollback (journal, snapshot, undo-capable application interface). Discipline: stage, witness the staged state, commit on match, roll back on mismatch; the rollback event itself is receipted.
- **observed_effect.** The target offers neither atomicity nor rollback (external device state, legacy application, already-dispatched message). Discipline: the effect is executed only after the strictest applicable validation, and the resulting state is observed and witnessed; divergence cannot be rolled back and is therefore surfaced as a deviation with the divergence itself sealed into the receipt. Policy MAY require a higher minimum Consent Mechanism Class (IX.18.3) for observed_effect transitions precisely because they are the least reversible.

IX.21.4 Class Declaration

The Transition Class is declared per effect in the Finalizer's effect schema, is part of the consented definition, and is stated in the client-generated preview (IX.5.3) — the operator sees, before consenting, whether the effect is atomic, reversible by rollback, or irreversible-observed.

IX.21.5 Declared Chains — the WCR Principle for Computer-Use

The executable computer-use chains — sequences of enveloped transitions — SHALL be defined in advance, per handshake or per deployment catalog admission, exactly as WCR recordings are for the web surface (IX.8.1): pinned, hash-identified, consent-admitted, version-suspended on change. No action is synthesized at runtime; a model participating in computer-use execution selects among declared chains, binds typed parameters within declared bounds, and chooses declared branches — the router discipline of IX.16.4 verbatim. An operation outside the declared chain set is unrepresentable as an execution request; an attempt is a deviation (IX.6.5).

IX.22 Relation to Operating-System Security Mechanisms (Informative)

The WR TED is deliberately implementable on top of existing platform security machinery; the mapping clarifies roles without binding any implementation:

- **Mandatory access control frameworks** (LSM-based systems such as SELinux and AppArmor) can serve as the enforcement substrate confining the executing components to their declared targets — enforcement, not trust anchor.
- **Measured integrity mechanisms** (IMA/EVM-class measurement) can supply witness corroboration for pre-/post-state claims on files and executables.

- **Hardware trust anchors** (TPM-class modules, platform keystores) serve as trust anchors: receipt-signing keys held in hardware (IX.10(1)), and C4 attestation evidence (IX.18.1).
- **Kernel observation frameworks** (eBPF-class tracing) can implement the witnessing of observed_effect transitions and deviation detection.
- **Platform audit frameworks** (auditd-class logging) provide a corroborating log beside — never instead of — the sealed receipt chain.

The feasibility staging is explicit: on today's platforms the WR TED composes from these mechanisms with differing depth per OS; a future dedicated runtime could carry the same normative semantics natively. Neither changes any rule of this block — which is the content of the definitional sentence: the WR TED is not an operating system, but a cryptographically governed execution authority that may be implemented using existing operating-system security mechanisms or future dedicated runtime environments.

IX.23 Claimed Contribution of the Execution Model (Informative)

The constituent mechanisms of this block are established practice and are not claimed as novel: transparency logs and countersigned inclusion proofs (certificate-transparency-class systems), public-ledger anchoring of hash commitments (blockchain notarization services), WebAuthn assertions and hardware-key second factors, TPM-class remote attestation, compare-and-swap transaction discipline, journaled and snapshot-based rollback, kernel-level observation, and append-only audit logs each exist independently. Per the claims discipline of IX.0.1, none of them is represented here as new.

The claimed contribution is the governed composition, as one contract from consent to proof: an unforgeable Execution Capability minted only by a consent event and attenuated to the consented space (IX.8.5); the Intent Hash binding the executed effect to the presentation the human actually confirmed (WYSIWYE, IX.19.2); the Consent Mechanism Class carrying the assurance level of that confirmation — policy-autonomous through hardware-attested — verifiably inside the proof itself (IX.18); a five-phase Finalizer lifecycle in which every consequence is validated, executed, pre-/post-state-witnessed, sealed into a hash-chained receipt, and only then anchored (IX.17, IX.19); three verification tiers in which anchoring is evidentiary and never validity-conferring, and nothing but hash commitments ever leaves the deployment (IX.20); the model confined to routing over pre-declared, handshake-pinned deterministic paths — selecting and parameterizing, never authoring (IX.16.4); and the generalization of the entire construction from contract-bound web automation to computer-use through typed Transition Classes and Transaction Envelopes, so that a file write, a process invocation, or a configuration change is proven under exactly the discipline of a web-surface Finalizer commit (IX.21). Individually, these ideas have neighbours; to the author's knowledge, no existing system composes them into a single consent-bound execution-and-proof chain of this form.

§IX.24 Version History

(Renumbered from §IX.16 in v2.0; content of all prior entries unchanged.)

Version	Date	Change
1.0	2026-07-14	Initial normative specification: the initiation layer — entry exclusively by WR Code® scan or manual code entry with invitation messages as untrusted code-distribution channels (code plus explainer only, no capsules, no formation links); invitation classes public_bearer (open redemption, identity lock at formation via both endpoints' stable identity fingerprints, no pre-formation recipient binding) and targeted_bound (reserved); single Hash-Pinned Consent covering handshake plus first declared automation; standing relationship under the grant model; transport-privacy tiers (sealed-relay default, disclosed P2P opt-in, split-hop and transport-level Obfuscation Mode reserved for High-Assurance); consent-gated escalation to full-context classes; reserved Credential Attachment envelope; and contract-bound web interaction with claims discipline (by-construction properties vs. objectives pending validation; no High-Assurance claim); Public Handshake with Hash-Pinned Consent, the client-generated preview contract, and origin binding, dual-channel publisher verification (DNS Discovery Record plus TLS origin channel, both agreeing with the append-only Assessment Record Store; discovery never authority);

Version	Date	Change
		two-object contract model — long-lived Handshake Contract carrying the Scope Envelope and grant-model rights, versioned Site Manifest with monotonic anti-rollback versioning and signed freshness with fail-closed offline semantics; change classes (silent/notice/consent) generated by the principle that consent attaches to the Scope Envelope and to automation definitions, never to content, with the Secure Browse tier split for structural growth; the WR View as a strictly passive, subset-pinned Presentation Surface rendered exclusively from a hash-verified local store with render-time network starvation; upstream media normalization on Optirando™ infrastructure with WebP as pinned raster serialization and hash-before-decode, video excluded in v1; typed declared slots as the sole dynamic-content mechanism; deviation hard-stop rule; WR Links as declared, consent-gated in-handshake triggers with client-generated previews from bound definitions; WCR Manifest Version Pinning with fail-closed suspension and silent-vs-consent re-enable; engine-agnostic isolation and key-custody invariants including the open-web-yields-no-WR-authority rule stated in advance of Profile B; append-only evidence model; implementation profiles (A:

Version	Date	Change
		system-engine WR-only client; B: full-engine browser product) as informative; staged roadmap and open specification items.
1.1	2026-07-15	Architectural clarifications, strictly additive — no existing rule weakened, no section renumbered: SHA-256 made explicit as the Pinned Digest Algorithm with a dedicated integrity-verification section (new IX.4.4: digest generation at approval/normalization, unconditional verification before processing on every render and replay path, rejection semantics, and the claims-discipline framing — cryptographic detectability and manifest-enforced integrity, no “tamper-proof” claim), cross-referenced from IX.4.2, IX.8.1, and IX.12.1; the WR Trusted Execution Domain and Pinned Digest Algorithm added as defined terms (IX.2); the deterministic execution model stated as a composed normative property (new IX.6.7: capability exclusions enumerated including timers and asynchronous execution interfaces — also added explicitly to the IX.6.1 exclusion list — the two-part no-evolution/manifest-confinement property, structural attack-surface reduction, reproducibility with determinism as defined in IX.8.2); manifest governance of rendering made explicit in IX.4.2 (the site is source,

Version	Date	Change
		never authority; post-approval site changes have no effect until a new version passes verification and the consent boundary); modular separation of pipeline stages raised to an engine-agnostic normative invariant (new IX.10(5): preparation / normalization / manifest generation / manifest verification / policy enforcement / rendering as distinct components, rendering as a replaceable module contributing no verification decision, the renderer explicitly not the primary security boundary, renderer hardening as defense in depth), reflected in the IX.13 profile framing; IX.12.1 by-construction properties extended accordingly (SHA-256 verification before processing, recording rejection on digest/pin failure, no post-verification evolution within the domain); motivation section extended with the deterministic-transformation framing (IX.1, informative); and an architectural outlook added as informative subsection IX.14.1 — the Hash-Bound Website & Automation Builder (WR Builder, name registered): deterministic canonical build artifacts produced alongside WCR recordings in one build pass, SHA-256-bound, manifest-referenced, verified by the client before execution, extending agreement on the

Version	Date	Change
		expected rendering state toward agreement on the exact executable code authorized to produce that state, with hash verification explicitly necessary-never-sufficient and PoAE™ semantics unchanged.
1.2	2026-07-15	Additive revision: the WR Compatibility Path introduced as a normative publisher-side mechanism (new IX.6.8, definition in IX.2, cross-referenced from IX.6.3 and IX.14.1, declaration format added to the open specification items of IX.15) — a dedicated representation authored directly for the Pinned Rendering Subset for sites that do not remain functional under transformation, traversing the identical normalization / SHA-256 / manifest / change-class / network-starvation pipeline with no relaxation of any rule, functional equivalence remaining the publisher's responsibility per IX.12.4, and WR Desk™ obligated to publish the authoring specification; the WR Builder outlook (IX.14.1) refined with the container boundary — the deterministic automation emitted in one build pass, together with its WCR recording, as an openly specified canonical container (publicly inspectable and auditable on the public path per Annex IV §IV.6.3(1)), SHA-256-bound and manifest-referenced with verification before execution, while

Version	Date	Change
		everything outside the container traverses ordinary normalization (media replaced by visually and informationally equivalent derivatives carrying no original byte structure) — exactness spent only where execution demands it; and a Relation to Prior Work subsection added (new IX.14.2) naming the closest neighbours — Subresource Integrity, Content Security Policy, Signed HTTP Exchanges / Web Bundles, AMP-class publisher parallel formats, browser reader modes, reproducible builds / binary transparency / code signing, and web archiving formats — with the claimed contribution stated as the governed, consent-bound composition. No existing rule weakened; no section renumbered.
1.3	2026-07-15	WR Builder outlook (IX.14.1) refined to a component-composition model: the builder conceived as a web builder with a prefabricated, WR-conforming component library — forms and registration flows in particular, interaction patterns, automations — each component canonical, openly specified, and hash-bound; publishers compose and parameterize instead of authoring free executable code; the complete composition, including the

Version	Date	Change
		exact executable code of every component instance, is summarized in the signed Site Manifest by SHA-256 digest and verified by the WR Execution Client before execution, with only matching code executing; the composition-not-synthesis parallel to Annex VIII §VIII.3.3 stated explicitly (the constrained executable space as the security floor, mirroring the role of layout grammars in VIII.5.3), with prefabricated forms as the canonical return path for interactive functions removed by strict passivity; component-based site builders added to the prior-work neighbours (IX.14.2) as the closest neighbour of the component model. No existing rule weakened; no section renumbered.
1.4	2026-07-15	WR Builder outlook (IX.14.1) extended with the open extension path and the closed action set: alongside the prefabricated component library, a publisher may integrate any arbitrary automation — including free, hand-written code — through the same build pass, the runtime being deliberately indifferent to authoring provenance and interested exclusively in the manifest and its enforcement; completion under either path yields the identical artifact discipline (exact executable code and precise execution sequence in

Version	Date	Change
		the signed Site Manifest, SHA-256-bound, verified before execution); and, decisively, the openness of the authoring path never widens the execution surface — regardless of what is authored, the execution path effects only actions from a closed, declared action vocabulary enforced at the renderer/execution boundary (per IX.8.2, under the register discipline of Annex VIII §VIII.6.5.3: “arbitrary action” never becomes a vocabulary entry), with out-of-vocabulary operations unrepresentable as execution requests and attempts hard-stopping per IX.6.5; the enforcement stack stated as hash verification (which code), action vocabulary (what it can effect), PoAE™ consent (whether it runs), preserving the client-generated preview contract for free code. The container point of IX.14.1 additionally names the precise execution sequence as manifest-bound. No existing rule weakened; no section renumbered.
1.5	2026-07-15	Problem statement and adoption revision, additive: IX.1 restructured into an explicit problem statement (IX.1.1 — the unknown-publisher trust problem and the failure of existing trust anchors for it), a normatively cross-referenced threat-to-mechanism mapping (IX.1.2

Version	Date	Change
		— undeclared code paths, undeclared resources, exfiltration, out-of-scope profiling, injection-based scope expansion, malware, unauthorized automations, out-of-contract execution, each mapped to the mechanism that addresses it under the claims discipline), the preserved platform context (IX.1.3, text unchanged), and the trust inversion with the consequence boundary (IX.1.4 — trust derived from cryptographic agreement rather than publisher reputation; the free decision space ends exactly where externally observable consequences begin; contracts declare boundaries — typed slots, declared action/read/write/transfer sets, the Scope Envelope — rather than enumerating runtime values; intelligence and automation as separate domains whose combination changes nothing at the boundary); governance-level cryptographic agility stated in IX.4.4 (algorithms replaceable without change to the trust architecture, hash-based integrity emphasized for post-quantum robustness); and a publisher adoption path added (new IX.14.3) — a four-level, strictly additive integration ladder from tool-generated read-only manifests (no code change) through annotated slots/links and WR-Builder-

Version	Date	Change
		composed automations to the dedicated representation, with manifest generation as a repeatable build step and the invariant that levels vary publisher effort, never client-side enforcement. No existing rule weakened; no section renumbered.
1.6	2026-07-15	The consequence-boundary execution model, additive (IX.8 retitled accordingly; new IX.8.3–IX.8.8; new definitions Finalizer, Decision Space, Decision-Space Consent, Execution Capability, Action Proposal, Plausibility Verification): three execution classes — Replay (pinned WCR, model as path-tolerance compensation, deterministic at the tool boundary), Selection (model selects within a manifest-declared Decision Space of enumerated options and typed bounds, informed by declared data sources whose values inform and never authorize), Proposal (always individually consented) — with class invariance (identical consequence boundary regardless of path intelligence), out-of-space divergence discipline (no execution below threshold, surfacing over improvisation), and the Annex VIII symmetry noted; the Finalizer as the non-negotiable deterministic enforcement point — manifest-declared, hash-verified under the root before any capability is minted

Version	Date	Change
		against it, version-bound in every proposal with anti-rollback, verifying effects (schema, bounds, atomic expected-state compare-and-swap, capability and consent evidence, idempotency by proposal hash, freshness) rather than code, complete over all externally observable consequences including reads and transmissions, returning signed receipts into the evidence chain; consent modes (per-action; Decision-Space Consent with the space itself hash-pinned in the preview) minting attenuated, sender-constrained Execution Capabilities held exclusively by the client core — selection latitude never carries execution authority; Context Admission / the Closed Model Context (three enumerated source classes, consent-gated source admission, non-admitted sources unrepresentable, injection unaddressable by construction where all sources are instruction-incapable, declared free-text sources consent-covered) with the IX.1.2 injection row sharpened accordingly; Plausibility Verification (rule-based, history-based, advisory model-based) under the ratchet rule — friction only ever increases, tiers modulate upward, no silent repair, dual-sided and mutually untrusting, verdicts in evidence with meta-

Version	Date	Change
		feedback dampening; the Effect-Binding principle (the agreement binds the effect, not the code: compromise yields at most failure to produce the agreed effect, never a non-agreed effect; hash pinning defends the honest path, Effect Binding the compromised path) with a corresponding new threat row in IX.1.2, new by-construction properties in IX.12.1, and the Finalizer implementation named as trust assumption (IX.12.3) and pending objective (IX.12.2); the WR Builder reframed as action-contract builder (IX.14.1); open specification items extended (Decision Space language, capability format, Finalizer schemas and receipts, state-binding profile, plausibility rule language, signature envelope). No existing rule weakened; no section renumbered.
1.7	2026-07-17	The link-as-entry failure class made explicit, additive: new informative subsection IX.1.5 stating the problem (a hyperlink fuses presentation, navigation, and claimed authority into one action, preventing a pre-contact trust boundary), the attack surface (publisher/destination spoofing, phishing invitations, misleading and homoglyph hostnames, redirect-chain manipulation, compromised distribution surfaces, link substitution and recontextualization, social

Version	Date	Change
		engineering, malware delivery, channel/authority confusion, habituation), the email case with the hidden-verification-path critique (SPF/DKIM/DMARC authenticate the sending domain via machinery reachable only to technical operators with time; WR replaces operator-performed visual or hidden-path verification with unconditional client-performed cryptographic verification whose success gates the consent interface), the architectural cause (the untrusted distribution channel permitted to claim authority), the four-phase control (distribution / entry / verification / consent) with distribution-is-not-entry as governing principle, manual code entry framed as a high-assurance property rather than a usability fallback, and the result (the threat class eliminated rather than mitigated, residue scoped per IX.12.4); a corresponding threat row added to IX.1.2 (handshake formation or authority claimed through a link; by construction); the canonical formulation “the code is an identifier and invitation — not authority; authority arises only from successful client-side cryptographic verification” made normative in IX.3.1 together with the explicit rule

Version	Date	Change
		that an accompanying hyperlink may inform or assist in locating the official client but SHALL NOT initiate, deep-link into, pre-fill, or otherwise shortcut the handshake flow; and the IX.12.1 entry-path property extended (codes untrusted until verification; no hyperlink forms, extends, or upgrades a handshake). No existing rule weakened; no section renumbered.
1.8	2026-07-17	Assisted Code Capture made normative, additive: IX.3.1 retitled (Entry Points, Capture Methods, and Code Distribution) and restructured around three capture methods — WR Code® scan, manual entry, and the new Assisted Code Capture (defined term added to IX.2) — converging on one identical formation path (verification chain, client-generated preview, Hash-Pinned Consent), with the invariant stated as capture-confers-no-authority / detection-is-not-entry / consent-completes-entry; assisted capture confined to two source classes — assisted_email (a WR code extracted from received message content by the depackaging pipeline, extraction running in the untrusted depackaging context on desktop with only the typed, bounded code token crossing into the client core) and assisted_discovery (WR

Version	Date	Change
		support detected via a Discovery Record on a visited or referenced domain) — under six rules: offer-never-action (a Connect offer in client chrome, only after successful verification, conferring no authority), unconditional fail-closed verification with the offer suppressed entirely on failure (no connect-anyway path), instruction-incapable extraction (the IX.8.6 admission discipline applied to capture: surrounding free text, markup, and links never cross the boundary), channel passivity (no navigation event or channel-delivered mechanism triggers, parameterizes, or shortcuts capture), provenance display and recording (preview, Handshake Contract per extended IX.4.1, evidence log), and policy governance (Secure Browse may disable or downgrade to notice-then-manual-confirmation; scan and manual entry never removable, manual entry retaining its IX.1.5 high-assurance role); IX.3.5 clarified (a Discovery Record may trigger assisted capture; a verified Connect offer is not auto-formation); the channel-prohibition sentence of IX.3.1 clarified to bind the channel, not the client; IX.1.5 extended with the rationale that assisted capture preserves the four-phase separation because

Version	Date	Change
		phase boundaries are drawn around authority, not effort (the client, never the channel, recognizes; the offer follows verification where a link precedes it; the residue — an attacker presenting its own correctly verified identity at lower friction — is unchanged and policy-bounded); the IX.1.2 link-threat row and the IX.12.1 formation-path property updated accordingly. No existing rule weakened; no section renumbered.
1.9	2026-07-17	Channel Provenance Verification made normative within Assisted Code Capture, additive: new rule 7 in IX.3.1 and new defined term Channel Provenance Record (IX.2) — the depackaging pipeline evaluates established channel-authentication standards at depackaging time (SPF, DKIM, DMARC, and presence/consistency of a Discovery Record for the authenticated sender domain) and emits a typed, bounded verdict record alongside the extracted code token, crossing the boundary under the same instruction-incapable discipline as the code (raw headers, signatures, and message content never cross); the Connect offer and consent preview state the verdicts as facts, including sender/publisher alignment against the verified publisher's bound origin set, enabling a fact-based operator decision

Version	Date	Change
		whether a Public Handshake should be formed from the channel at all; verdicts governed by the ratchet discipline of IX.8.7 applied to capture (inform only; may suppress, downgrade, or annotate the offer — never upgrade trust, reduce friction, pre-satisfy consent, or substitute for any step of the WR verification chain, whose validity is channel-independent), with the claims discipline stated (channel standards authenticate the sending domain, not publisher intent or code safety; the structural guarantee — a hostile publisher can effect nothing beyond the verified contract — does not depend on them, and a passing channel is not evidence of a trustworthy publisher); record retained in the evidence log; IX.1.5 extended (the hidden verification path of email surfaced automatically at the decision point; verification chain answers what can execute, channel verdict answers whether to form at all); Secure Browse extended with minimum channel-provenance requirements for assisted_email offers as a ratchet-conforming policy constraint (IX.9); and a corresponding by-construction property added to IX.12.1 (channel provenance informs and never authorizes). No existing rule weakened; no

Version	Date	Change
		section renumbered.
1.10	2026-07-17	Uniform surfacing of channel verdicts, additive: new rule 8 in IX.3.1 — Channel Provenance Verification decoupled from code presence (the depackaging pipeline evaluates SPF/DKIM/DMARC and Discovery-Record consistency for every processed message and retains the Channel Provenance Record with the depackaged message), with the record surfaced under identical verdict semantics and identical alert presentation at every channel-consequential decision point, minimally the Connect offer / consent preview and the external-link risk dialog presented before any external link carried in a depackaged message is opened; where DKIM and DMARC are absent or unverifiable, both surfaces display a prominently highlighted red alert-class warning (identical styling) stating that the sender address’s authenticity could not be verified and that links and codes in the message therefore have no verified origin — the warning states unverifiability, not maliciousness, is ratchet-conforming, and SHALL NOT be suppressible by message content, publisher content, or any non-policy mechanism, while Secure Browse may escalate it up to blocking

Version	Date	Change
		external-link opening from unauthenticated channels but never remove it (one evaluation at depackaging, one record, one presentation, every decision point); the Channel Provenance Record definition generalized accordingly (IX.2: emitted for every processed message, retained with it, consumed by capture and by the link dialog); IX.1.5 extended (the record follows the message beyond capture; the unverifiable-sender condition is flagged in red wherever the operator acts on the message's content); the IX.12.1 channel-provenance property extended (uniform surfacing, unsuppressible warning); and the Secure Browse policy hooks of IX.9 extended (external-link escalation; the warning itself never removable by policy). No existing rule weakened; no section renumbered.
1.11	2026-07-17	Scam-analysis integration, additive: new rule 9 in IX.3.1 — the Channel Provenance Record is provided as a declared, typed input to the deployment's local scam/phishing analysis component (Fraud Watchdog: the WR Desk™ auto-run analysis over validated message text, sender metadata, and link strings, on host and sandbox, never touching original artifacts, opening attachments, or following links), whose phishing-risk

Version	Date	Change
		assessment incorporates the deterministic channel verdicts — strengthening in particular the brand-impersonation cross-check (brand claim + action request + unauthenticated or publisher-unaligned sender as a materially stronger composite finding); the layering fixed as strict and one-directional (rule-7/8 verdicts are facts, deterministic and unconditionally presented; the Watchdog is heuristic interpretation over facts, advisory under the ratchet discipline of IX.8.7(1)(c) applied to message analysis), with findings surfaceable at the rule-8 decision points visually distinguished from verified facts, permitted to escalate or annotate, and prohibited from suppressing, softening, preceding, or replacing the deterministic unverifiable-sender warning, upgrading trust, or reducing friction — a benign heuristic assessment never removes the deterministic warning; analysis local-only consistent with IX.11 (no off-device transmission of record or content for this purpose); claims discipline stated (heuristic, false-positive-averse, advisory, not verification) and the component's operational safety added to the pending-validation objectives (IX.12.2); Channel

Version	Date	Change
		Provenance Record definition (IX.2), IX.1.5, and the IX.12.1 channel-provenance property extended accordingly. No existing rule weakened; no section renumbered.
1.12	2026-07-17	Fraud Watchdog established as canonical designation, additive: Fraud Watchdog added as a defined term (IX.2) — the internal designation of the WR Desk™ local scam-analysis mode whose purpose is to make fraud, phishing, and social-engineering attempts locally visible through analysis (auto-run over validated message text, sender metadata, link strings, and the Channel Provenance Record; host and sandbox; no original artifacts touched, no attachments opened, no links followed, nothing transmitted off-device; findings heuristic and advisory under the ratchet discipline, never carrying, upgrading, or substituting for verification or authority), with the name registered to prevent divergent usage; IX.3.1 rule 9 retitled Fraud Watchdog integration and reworded to reference the registered designation. No existing rule weakened; no section renumbered.
1.13	2026-07-17	Depackaging model corrected and brand designations fixed, corrective and additive: the assisted_email description of IX.3.1 corrected — depackaging transforms the received email into the BEAP™ format (machine- and

Version	Date	Change
		human-readable, text-pure under allowlist positive construction, secure and deterministic by design), the original artifact never crossing the boundary (blob-inertness), the WR code recognized within the validated BEAP™ text, and only the code token parameterizing the capture flow (superseding the v1.8 formulation “only the code token crosses the boundary”, which understated what depackaging yields); rule 3 retitled Instruction-incapable parameterization and reworded accordingly (the BEAP™ text is ordinary safe message content under the text-purity discipline; it and its links never trigger, parameterize, or shortcut the flow); rules 7–8 reframed to the one-transformation model — channel standards (SPF, DKIM, DMARC, Discovery Record) evaluated automatically within the BEAP™ depackaging transformation itself, the Channel Provenance Record carried as typed fields of the resulting BEAP™ message and traveling with it to every decision surface (Connect offer, consent preview, external-link risk dialog), raw headers and signatures remaining with the original artifact; Fraud Watchdog™ adopted as product designation (trademark notice extended; IX.2 definition, rule 9, IX.1.5,

Version	Date	Change
		and IX.12.2 updated to the marked form); Assisted Code Capture and Channel Provenance Record definitions (IX.2) aligned to the corrected model. No security property weakened — the corrected model is strictly stronger in description and identical in enforcement; no section renumbered.
2.0	2026-07-18	The WR Trusted Execution Domain and the Finalizer Execution Model, additive (new section block IX.16–IX.23; Version History renumbered from §IX.16 to §IX.24 — the only renumbering, affecting no content section): scope extended to the execution semantics of the WR TED and its generalization from contract-bound web interaction to computer-use; WR TED characterized as execution authority, not operating system (“not an operating system… cryptographically governed execution authority… existing operating-system security mechanisms or future dedicated runtime environments”), with domain responsibilities enumerated and the AI characterized normatively as a router over pre-declared, handshake-pinned deterministic paths (selects, parameterizes, branches — never creates, extends, or modifies a path; every route halts at every

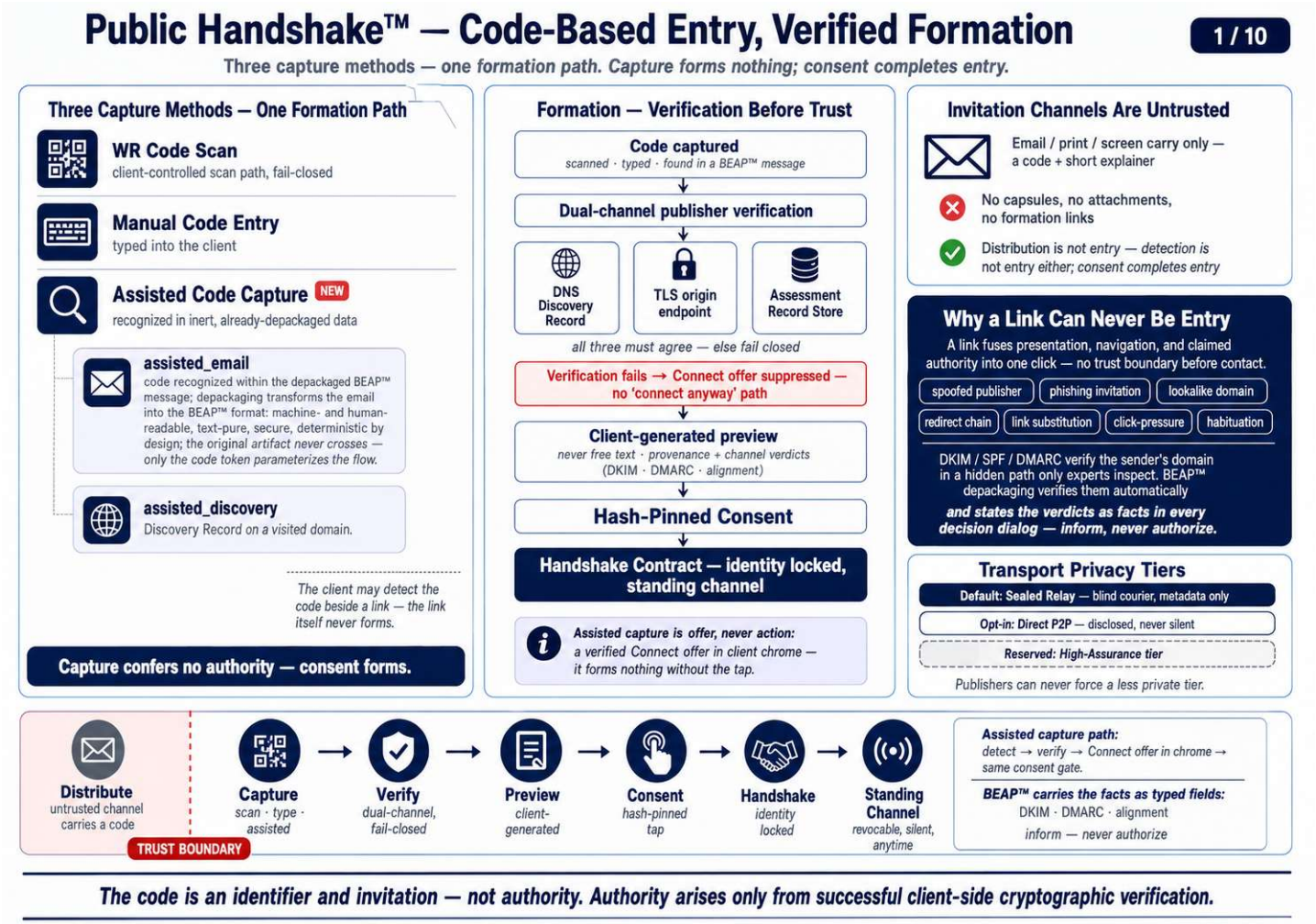
Version	Date	Change
		PoAE™ gate; “the AI routes; it never paves”); the Finalizer execution lifecycle made explicit as five normative phases Validate → Execute → Witness → Seal → Anchor, phases 1–2 restating IX.8.4 unchanged, witnessing binding pre- and post-state hashes of every declared target, sealing completing validity, anchoring asynchronous and never validity-conferring; Consent Mechanism Classes C0–C4 (policy-autonomous with consent-admitted policy rules, UI confirmation, Passkey/WebAuthn with the assertion as in-record evidence, hardware-security-key 2FA, hardware attestation) under a meet-or-exceed rule with policy-declared minimums per PoAE™ tier, ratchet-only escalation, and the class artifact bound into the proof; the PoAE™ record / Execution Receipt field list specified in full including the Intent Hash (What-You-See-Is-What-You-Execute binding between the confirmed presentation and the executed effect, closing the display-versus-execution gap of agentic systems), Transition Class, monotonic sequence number, and previous-receipt hash forming an append-only local hash chain; Verification Tiers L / C / X — local hash chain, countersigned central transparency log on Optirando™ infrastructure

Version	Date	Change
		(hash commitments only), external Merkle-root anchoring with Solana as named ledger-agnostic embodiment — with tier invariants (validity at Seal; anchoring evidentiary only; digests only off-device; higher tiers add verifiability, never authority; publishers can force neither lower nor extra-contractual higher tiers); the Transaction Envelope for computer-use with typed Transition Classes atomic / journaled / observed_effect, pre/post-state witnessing, class statement in the consent preview, per-class consent-minimum policy, and the WCR principle carried over (chains declared in advance, runtime selection only, no action synthesis); an informative mapping to OS security mechanisms (MAC frameworks, measured integrity, hardware trust anchors, kernel observation, audit frameworks — enforcement vs. trust-anchor vs. witness roles, feasibility staged); a claimed-contribution statement for the execution model under the claims discipline (IX.23), cross-referenced from IX.14.2; corresponding by-construction properties, pending objectives, and trust assumptions added to IX.12.1–IX.12.3; new defined terms (generalized WR TED, Intent Hash, Transition Class, Consent Mechanism Class,

Version	Date	Change
		Verification Tier, Transaction Envelope — IX.16.2, pointer in IX.2); and corresponding open specification items appended to IX.15. No existing rule weakened; no content section renumbered.

License Notice

This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.



BEAP™ Channel Provenance + Scam Watchdog™ – Facts Before Interpretation

2 / 10

One evaluation at depackaging, one record, one presentation — every decision point.

One Transformation — BEAP™ Depackaging

Received email

BEAP™ depackaging transformation
untrusted context — desktop: microVM

- SPF
- DKIM
- DMARC
- WR Discovery Record of the sender domain
- sender/publisher alignment

Channel Provenance Record — typed fields carried with the BEAP™ message

BEAP™ message — machine- and human-readable · text-pure · secure · deterministic by design

The original artifact never crosses the boundary (blob-inertness); raw headers and signatures stay with it. Verdicts travel with the message by design.

The Hidden Path, Surfaced

Email buries its verification in headers only experts can excavate. BEAP™ evaluates it automatically and states the verdicts as plain facts at the decision point — for non-technical operators under time pressure.

One Record — Every Decision Point

Channel Provenance Record

- **Connect Offer / Consent Preview** — *verdicts + sender/publisher alignment shown as facts*
- **External-Link Risk Dialog** — *identical verdicts, identical alert styling, shown before any external link from the message opens*
- **Scam Watchdog™** — *declared, typed input into local phishing-risk analysis*

DKIM and DMARC absent — the sender address cannot be verified. Links and codes in this message have no verified origin.

Identical red alert-class warning on both surfaces · unsuppressible by message or publisher content · policy may escalate (up to blocking link opening) — never remove.

Facts vs. Interpretation — One-Directional Layering

Facts — rules 7–8
deterministic channel verdicts, presented unconditionally.

Interpretation — Scam Watchdog™
heuristic, model-based, advisory (ratchet discipline).

- Findings may add warnings and escalate presentation
- Findings never suppress, soften, or replace the deterministic warning
- A benign assessment never removes the unverifiable-sender warning.

Scam Watchdog™ — Locally Visible Fraud

The WR Desk™ local scam-analysis mode: makes fraud, phishing, and social-engineering attempts locally visible through analysis.

- *analyzes validated text, sender metadata, link strings + the Channel Provenance Record*
- *runs on host and sandbox*
- *touches no originals, opens no attachments, follows no links*
- *transmits nothing off-device*
- *false-positive-averse*

Sharpened cross-check: brand claim + action request + unauthenticated or publisher-unaligned sender = a materially stronger phishing finding than any signal alone.

Depackage — *email → BEAP™*
→ **Evaluate** — *SPF · DKIM · DMARC · Discovery*
→ **Record** — *typed verdict fields*
→ **Surface** — *offer · preview · link dialog*
→ **Analyze** — *Scam Watchdog™*
→ **Decide** — *the operator, on facts*

Channel verdicts inform and never authorize — analysis can add warnings on top of the facts, never argue them away.

The Two-Object Contract — Consent That Stays Accurate Forever

3 / 10

Consent attaches to the Scope Envelope and to automations — never to content.

Why Two Objects

One object would force re-consent for every content change
consent fatigue destroys the signal

Or silent mutation of a consented object
the record stops describing what runs

Two objects keep every consent record permanently accurate
authorization stays durable while content evolves

Publisher Root Key

signs ↓ signs ↓

Handshake Contract — long-lived

1. Publisher identity + attestation level
2. Bound origin set — enumerated, no wildcards
3. Rights: delivery + preparation only
4. No-expiry-until-revoke, silently revocable
5. Scope Envelope — the outer boundary
6. Capture method + provenance recorded — scan · manual · assisted_email · assisted_discovery

Site Manifest — versioned content

1. Page set + structure (Pinned Rendering Subset)
2. Resource hashes — SHA-256
3. Typed dynamic-value slots
4. WR Link declarations
5. Automations by WCR recording hash
6. Decision Spaces + Finalizer declarations

Change Classes

- **CONSENT** — **Blocking, PoAE™-recorded changes** — new automation · new origin · envelope expansion · root key rotation
- **NOTICE** — **Visible, non-blocking changes** — relocated WR Links · re-signed previews at identical scope
- **SILENT** — **Verified on next use changes** — slot values · re-hashed resources · layout within subset

Anti-Rollback + Freshness

- **Versions strictly monotonic** — a validly signed stale manifest is the rollback attack
- **Signed freshness assertion**
- **Offline: last verified state, visibly stale, automations suspended**

Sign — Publisher signs both objects with root key
→ **Version** — Assign monotonic version numbers
→ **Verify** — Validate signatures, attestation, and freshness
→ **Classify Change** — Compare to last verified state and classify
→ **Notice / Consent** — Provide notice or request consent as required
→ **Render** — Render content within verified scope

Content moves. The consent record never lies about what was authorized.

The WR View — Exclusion Instead of Containment

4 / 10

The publisher's site is transformed upstream into a hash-bound, strictly passive representation.

WR Trusted Execution Domain — Eliminated Inside

- JavaScript
- WebAssembly
- Timers + scheduled callbacks
- Asynchronous APIs
- Unauthorized network
- Dynamic code paths

The rendered page cannot evolve beyond the approved manifest state.

Dynamic Content = Declared Slots Only

- Typed, bounded slots declared in the manifest
- Pure value binding — rendered inert, never markup
- Free HTML injection does not exist as a mechanism.

Content Pipeline — SHA-256 End to End

- Publisher Website
- Upstream Normalization (Optirando™ infrastructure — WebP derivatives, never originals)
- SHA-256 Hash Generation
- Signed Site Manifest
- Out-of-Band Fetch + Hash Verification
- Verified Local Store
- **WR View — zero network at render time**

render-time network **starvation:** an undeclared resource has no route to the network by construction

Presentation Integrity

- Nothing displays before validation
- No layout shift — reserved geometry
- Atomic reveal per validated region
- Deviation → detect, block, hard stop, inform, record

Operational Effect

- Injection + post-load substitution inexpressible
- Reproducible: bounded state + verifiable inputs
- Auditable: what was shown is provable after the fact

Normalize upstream (normalization pipeline) → **Hash** SHA-256 for every resource → **Sign** Site Manifest with publisher root key → **Fetch** out-of-band (on demand, no live links) → **Verify** hash verification against the manifest → **Store** verified local store (read-only) → **Render** zero network at render time

The desktop contains what it cannot exclude. The WR View excludes so nothing needs containing.

WR Links + WCR Recordings — Execution Without Improvisation

5 / 10

Displayed execution, permitted execution, and performed execution must coincide.

Manifest Version Pinning

- Recording pinned to a manifest version range
- Version bump touches a dependency
- Suspend — fail closed

- Definition unchanged → silent re-enable
- Definition changed → new consent

"Latest version" is not a valid pin. No automation silently adapts to a changed site.

WR Link Activation

1. Valid handshake — operator in the WR View
2. Operator activates a declared WR Link
3. Client loads the bound definition (never free text)
4. Client-generated preview: what executes, reads, transmits, changes, persists
5. Consent tap — PoAE™ record, the tap is the authorization
6. **Exactly the declared function executes — monitored**

Execution Semantics

- Only declared steps execute.
- Unknown action, unexpected state, changed resource, out-of-bounds value → hard stop + recorded divergence.
- Determinism = bounded state + action set with verifiable inputs — not identical pixels.

No Authority Anywhere

- A link confers none.
- A preview confers none.
- Manifest placement confers none.
- A captured code confers none — capture ≠ formation (new).
- **Only the human consent tap** with a PoAE™ record executes.

Declare define WR Link + bound WCR recording → **Pin** recording pinned to manifest version range → **Preview** client-generated execution preview → **Consent** PoAE™ record — the tap is the authorization → **Execute** only declared function executes → **Monitor** runtime monitoring + divergence → **Evidence** PoAE™ record + execution receipt

Propose, never silently apply — the web surface obeys the same law as the rest of the Canon.

By Construction vs. Pending Validation — The Claims Discipline

Two kinds of security statements, never mixed.

Properties by Construction

1. No undeclared resource fetchable at render time
2. No active content in a WR View
3. Nothing renders without a prior SHA-256 match against a signed, monotonic, fresh manifest
4. Modified recordings are rejected, never repaired
5. The WR View cannot evolve beyond the approved state
6. No automation outside declaration, pin, and consent
7. No formation besides the three capture methods — scan, manual entry, assisted capture — all gated by verification + consent; no hyperlink forms a handshake
8. Consent records permanently describe what was consented to
9. No consequence outside a declared, verified Finalizer — no call without a valid Execution Capability
10. Path intelligence carries no authority — a compromised path can at most withhold the agreed effect, never produce a non-agreed one
11. Injection unaddressable where every context source is typed and instruction-incapable
12. Channel provenance informs and never authorizes — verdicts surfaced at every decision point; the unverifiable-sender warning is unsuppressible.

Objectives Pending Validation

- Parser resistance to malformed input
- Normalization pipeline reproducibility
- Interception robustness across engine versions
- Behavior under hostile platform conditions
- Finalizer implementation: atomic, idempotent, replay-resistant
- Advisory plausibility stages safe under the ratchet rule
- Scam Watchdog™ detection quality + false-positive discipline in its advisory role

claimed only after implementation, testing, adversarial analysis, and audit.

Modular Architecture

replacable — contributes no verification decision

the renderer is not the security boundary.

Key + Capability Custody

- Keys, consent capture, and Execution Capabilities live only in the client core, hardware-backed
- A compromised rendering or reasoning context yields no WR authority
- Append-only local evidence log

Reduce Surface — eliminate what is not required. → **Verify Always** — cryptographic verification at every boundary. → **Isolate Keys** — keys + capabilities in the client core, hardware-backed. → **Log Everything** — append-only evidence for every consequential step. → **Prove Afterward** — reconstruct and verify what actually happened.

Reduced attack surface by construction, contained residue by platform — not invulnerability.

The Consequence Boundary — Free Inside, Bound at the Effect

The free decision space ends exactly where externally observable consequences begin.

Three Execution Classes

- **REPLAY** — pinned WCR recording; the model is path tolerance, deterministic at the tool boundary.
- **SELECTION** — the model chooses within a declared Decision Space: enumerated options, typed bounds, declared data sources; data informs, never authorizes.
- **PROPOSAL** — freely composed action, always individually consented.

Closed Model Context

Three source classes only:

- consent-admitted data sources
- operator inputs

- New sources join only by consent — never silently.
- Non-admitted sources: unrepresentable as context.

all sources typed = injection unaddressable by construction.

One Boundary for Every Class

1. **Action Proposal**
2. **Validation** — schema · bounds · Finalizer version
3. **Plausibility Verification** — rules · history · advisory model (ratchet: friction only ever increases)
4. **Consent** — per action or Decision Space; mints the Execution Capability
5. **Finalizer** — hash-verified before use; effect check: state compare-and-swap · capability · idempotency · freshness
6. **Signed Execution Receipt**
7. **Append-Only Evidence — PoAE™ chain**

Execution Capabilities

- Minted only by consent, attenuated to exactly the consented space.
- Sender-constrained, held only by the client core.
- No consent → no capability → no representable Finalizer call.

selection latitude, never execution authority — mechanically.

Effect Binding

The agreement binds the effect — not the code.

- Hacked code can at most fail to produce the agreed effect — never produce a non-agreed one.
- Hash pinning defends the honest path; Effect Binding defends the compromised path.

Propose — describe the intended action → **Validate** — schema, bounds, Finalizer version → **Plausibility** — rules, history, advisory model → **Consent** — mints Execution Capability → **Finalize** — effect execution with capability → **Receipt** — signed execution receipt → **Evidence** — append-only PoAE™ chain

The security statement is independent of the intelligence of the path.

Finalizers + Capabilities — Public Functions, Consent-Bound Effects

8 / 10

What may run is declared in the repo and bound in the Handshake or BEAP™; authority exists only after consent or a pre-admitted policy decision.

Declared Finalizer Classes

- **READ** — Fetch from a declared source; no new source admission
- **TRANSMIT** — Send a declared payload to a declared endpoint
- **CHANGE** — Mutate declared state in a local or remote system
- **PERSIST** — Write a declared record, file, or evidence item
- **EXECUTE** — Invoke a declared tool, workflow step, or adapter path

every callable effect function is public, hash-pinned, and vocabulary-bound.

What Lives in the Repo

- **Finalizers** — publicly inspectable effect functions
- **Capability schemas** — the exact effect space a token may authorize
- **LoRA adapters** — declared model adaptation assets
- **Exportable-session AI prompts** — prompt assets carried with the session
- **Normalized WR View libraries** — approved image/resource derivatives
- **WCR recordings + manifests** — inspectable execution declarations

the function surface is declared before any effect is allowed.

From Repo to Consequence

- **Repo Surface** — public functions, schemas, LoRA adapters, WR View libraries, exportable-session prompt assets (inspectable · hash-addressed)
- **Handshake / BEAP™ Agreement** — binds artifact references, allowed Finalizers, capability schema, and data scope
- **Consent / Policy Decision** — live operator consent or a previously admitted policy rule (policy may pre-authorize only inside declared bounds)
- **Execution Capability Token** — minted only here; sender-constrained and attenuated to the exact consented effect space
- **Hash-Verified Finalizer** — only the pinned repo function may execute (public code, private authority)
- **Signed Execution Receipt**
- **Append-Only Evidence — PoAE™ chain**

Agreement Discipline

- The function implementation is part of the declared repo surface
- The Handshake or BEAP™ pins which Finalizers may be called
- Data may inform a choice, never authorize a new effect
- No consent → no capability → no representable Finalizer call.

consent may be per action, per Decision Space, or policy-carried.

Execution Capabilities

Authority is never inferred from the model path.

- Minted only by consent
- Can authorize only the exact declared effect space
- Held only by the client core
- Forwarded, widened, or replayed tokens are invalid
- Freshness, idempotency, and sender binding are checked at Finalizer time.

selection latitude, never execution authority — mechanically.

Declare functions, schemas, adapters, libraries, prompts, manifests → **Agree** Handshake / BEAP™ binds what is allowed → **Consent / Policy** operator consent or pre-admitted policy → **Mint Capability** minted, attenuated, sender-constrained token → **Finalize** hash-verified function executes → **Receipt** signed execution receipt → **Evidence** append-only PoAE™ chain

Open functions in the repo; bound consequences at the edge.

WR Builder — From Rendering State to Executable Code

9 / 10

CONCEPTUAL FUTURE EXTENSION — NOT AN IMPLEMENTED FEATURE

The Conceptual Step

Today: agreement on the expected rendering state

↓

Outlook: agreement on the exact executable code authorized to produce that state.

What Lives in the Repo

- **Finalizers** — publicly inspectable effect functions
- **Capability schemas** — the exact effect space a token may authorize
- **LoRA adapters** — declared model adaptation assets
- **Exportable-session AI prompts** — prompt assets carried with the session
- **Normalized WR View libraries** — approved image/resource derivatives
- **WCR recordings + manifests** — inspectable execution declarations

the function surface is declared before any effect is allowed.

Prefabricated component library — forms · flows · automations

Free code — open extension path

same manifest, same enforcement

- **WR Builder** — one build pass
- **Canonical Build Artifact** — exact code, configuration, resources + precise execution sequence (produced alongside the WCR recording)
- **SHA-256 Hash Generation**
- **Signed Site Manifest**
- **Runtime Verification** — hash match required
- **Verified Execution — only matching code runs, whoever wrote it.**

The Closed Action Set

- The authoring path never widens the execution surface.
- Out-of-vocabulary operations are unrepresentable as execution requests.
- "Arbitrary action" never becomes a vocabulary entry.

hash verification = which code · action vocabulary = what it can effect · PoAE™ consent = whether it runs.

Everything Carries Over

- Modification → cryptographically detectable
- Execution → manifest-governed, default-deny
- Deviation → hard stop
- Security → the pipeline, not the executor

Compose declare user intent + select components within vocabulary → **Build** WR Builder — one build pass → **Hash** SHA-256 of the build artifact → **Sign** signed site manifest → **Verify** hash match required → **Execute** only matching code runs

The same manifest-driven trust model, extended one level deeper — when it can be carried in full.

WR Builder — Exact Where It Executes, Normalized Everywhere Else

One build pass, three zones: a verified container, a normalized surrounding, a governed fallback.

CONCEPTUAL FUTURE EXTENSION

1 — The Open Container (exact)

WR Builder — one build pass

WCR recording produced and bound alongside

Open-Source Container

- executable code
- configuration
- resources

openly specified — publicly inspectable

SHA-256 Hash

Signed Site Manifest

WR Desk™ Runtime — hash verification before execution

Deterministic Execution — only the matching container runs

byte-exact, because execution demands exactness.

THE CONTAINER BOUNDARY

2 — Everything Outside (normalized)

Ordinary site content around the container

Upstream Normalization

- media replaced by equivalent representations
- same look — new file, no original bytes

Minimal Normalized Representation in the WR View

equivalent, because rendering demands only equivalence.

3 — The Fallback (WR Compatibility Path)

Site breaks under transformation?

Publisher authors a second path in its own codebase — designed for the WR subset

Identical pipeline: normalize · hash · manifest · consent — no relaxation

WR Desk™ Runtime renders the fallback WR View

The model's strictness never bends — the accommodation lives on the authoring side.

Build compose code, config, resources + sequence → **Contain** package into the open-source container → **Normalize** transform everything outside to equivalent representations → **Sign** sign manifest with publisher root key → **Verify** fetch + verify hashes, schema, freshness → **Execute** render WR View or verified fallback WR View

Exactness is spent only where it is load-bearing — everything else is equivalent, and everything is governed.